Buildstaging.com

VAPT Report on Web Application and Network Infrastructure of Buildstaging.com

DATE: January 18, 2026

VAPT Project Report By
EHP Batch-9 (Team: Blue Diamond)
Byte Capsule IT
Farmgate, Dhaka
https://bytecapsuleit.com

Contents

Engagement Contacts

Pentesting Team (EHP-9 Blue Team)

Name	Title	Email	Phone Number
Md Mazadul Islam	Jr. Pentester (Team Leader)	pentester.mazadulislam@gmail.com	01746777198
Md. Al Mamun Siddique	Jr. Pentester	mamun.pentester@gmail.com	01711522247
Mahmuda Binte Kabir	Jr. Pentester	pentester.mahmuda@gmail.com	01780022376
Md. Mahir Faysal	Jr. Pentester	fmahir2015@gmail.com	01730631298
Md. Redwanul Karim	Jr. Pentester	pentester.redwan@gmail.com	01518971623

1. Executive Summary

This Vulnerability Assessment and Penetration Testing (VAPT) report outlines the security evaluation conducted on buildstaging.com, a staging site associated with Hotmart's digital products platform. The assessment focused on identifying vulnerabilities aligned with OWASP Top 10 risks, including broken authentication failures.

2. Introduction

Objective:

The primary objective of this VAPT was to identify security weaknesses in buildstaging.com that could lead to unauthorized access, data exposure, or service disruption. This demo simulates a real assessment to evaluate the site's resilience against common web attacks, focusing on reconnaissance, enumeration, and manual/automated testing.

Scope:

- **In-Scope:** buildstaging.com and subdomains (e.g., sso.buildstaging.com, nalytics.buildstaging.com).

Focused Areas: OWASP Top 10 Vulnerabilities

- **A07:** Identification and Authentication Failures.

3. Information Gathering

Passive Reconnaissance (Open-Source Intelligence – OSINT)

Objective: Collect publicly available information about the target without direct interaction.

Methodology:

- Search engines (Google, Bing) with advanced dorks
- Certificate Transparency logs (crt.sh)
- DNS enumeration (subdomain discovery)

- GitHub, public source code leaks
- Wayback Machine (archive.org)
- Technology fingerprinting via builtwith.com, wappalyzer patterns

Perform Google Dorks search:

I started by looking for open directory listings and sensitive backup files to see if the server configuration was weak. I didn't find any exposed directories or critical backup files in this initial check.

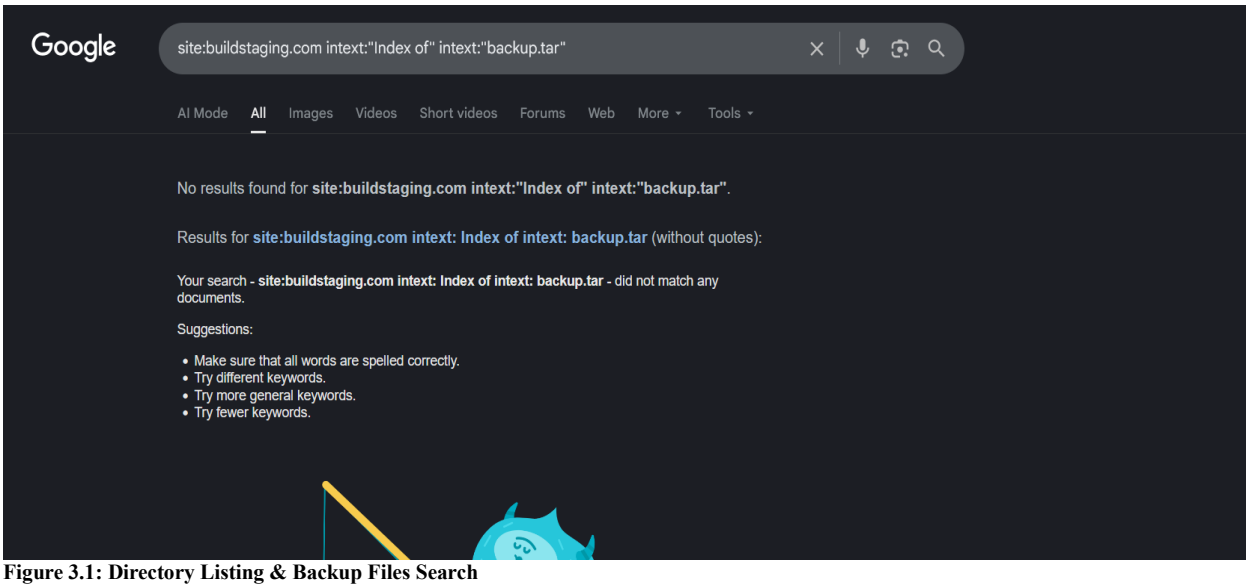


Figure 3.1: Directory Listing & Backup Files Search

Next, I searched for pages with specific parameters that are often susceptible to Authentication Failures such as login.php or viewid=. Similar to the first step, this didn't return any documents, meaning these specific vulnerable endpoints weren't indexed.

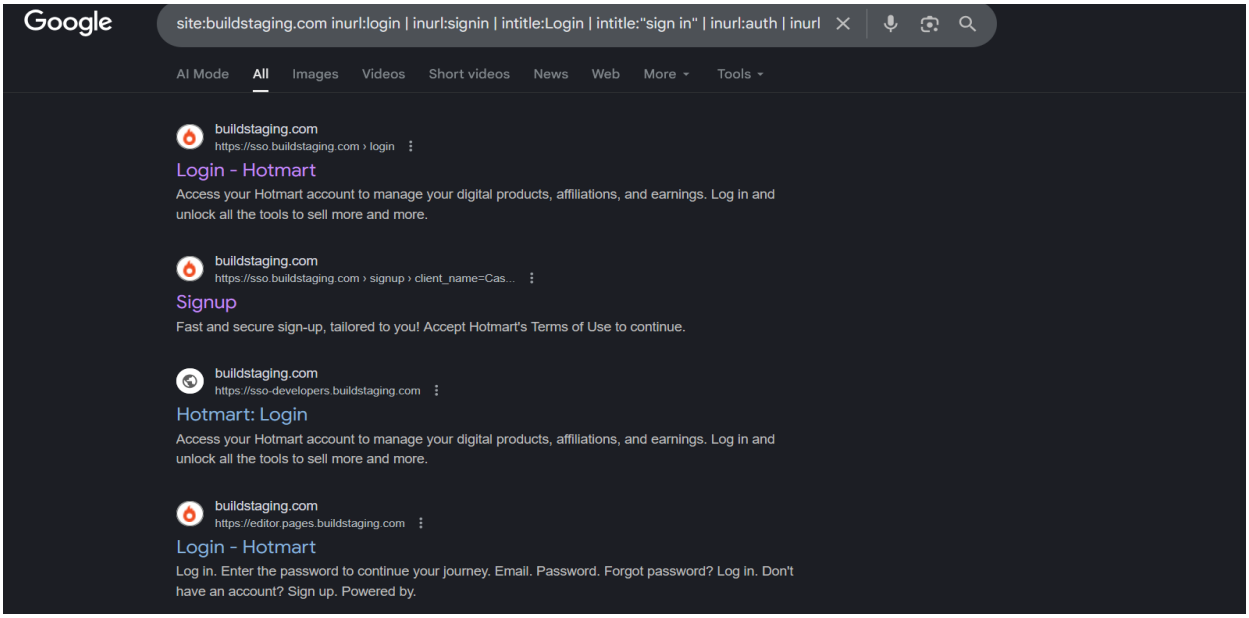


Figure 3.2: Vulnerable Parameters Search

Domain & Subdomain Enumeration:

- Main domain: buildstaging.com
- Discovered subdomains (via certificate logs & search engine results):

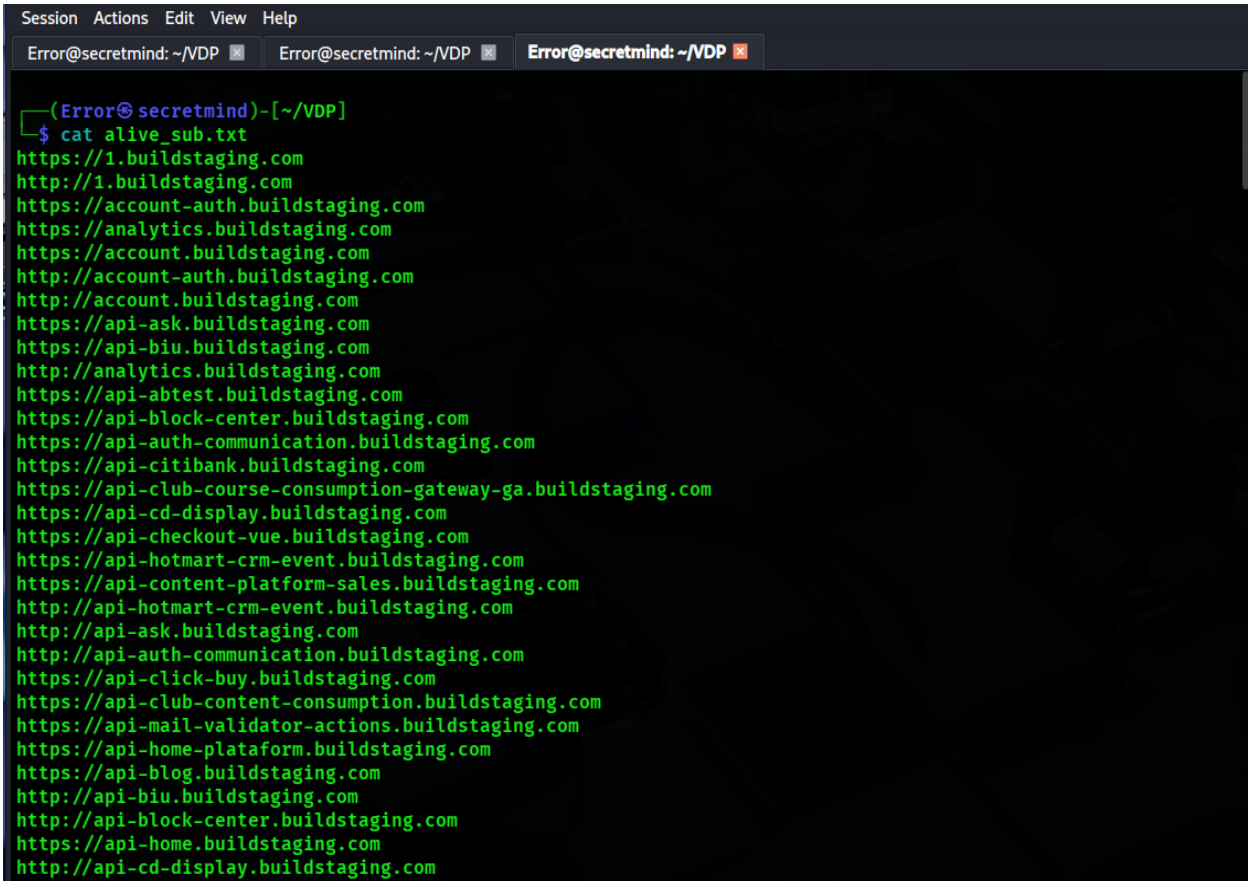


Figure 3.3: Discovered alive subdomains

Publicly Exposed Authentication Interfaces

- https://sso.buildstaging.com/login → Email + Password form → Contains terms acceptance checkbox → JavaScript required
- https://live.buildstaging.com → Very simple email/password login
- https://analytics.buildstaging.com → Separate dashboard login (different style)
- Multiple login pages with slightly different branding → Indicates possible fragmented auth implementation

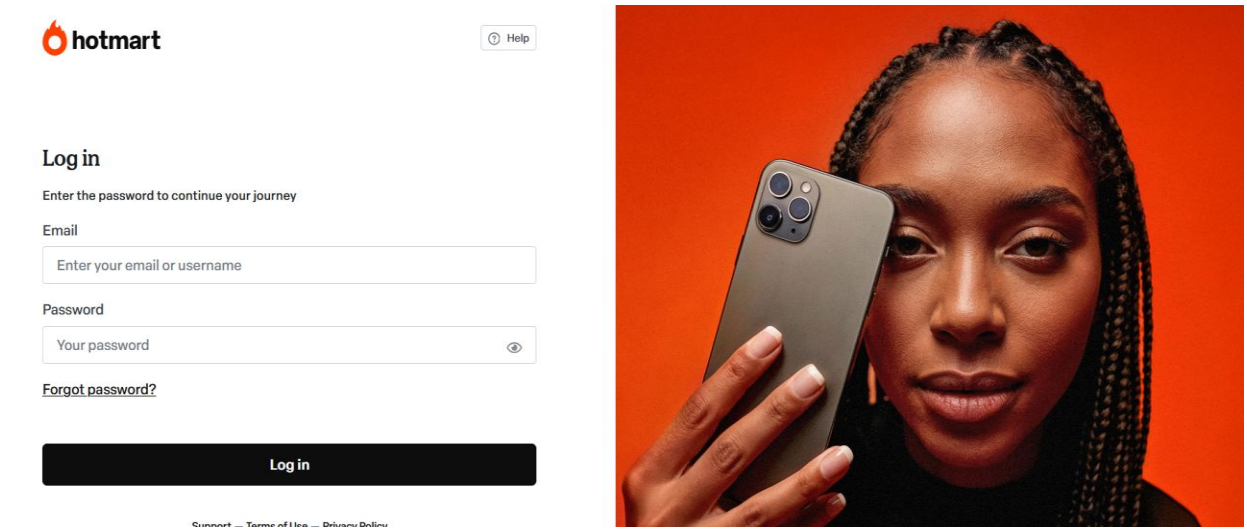


Figure 3.4: Publicly Exposed Authentication Interfaces

Historical Data (Wayback Machine):

- Very limited snapshots available
- No obvious historical leaks of .env, config files, or credentials
- Some old snapshots show /en/signup redirect behavior

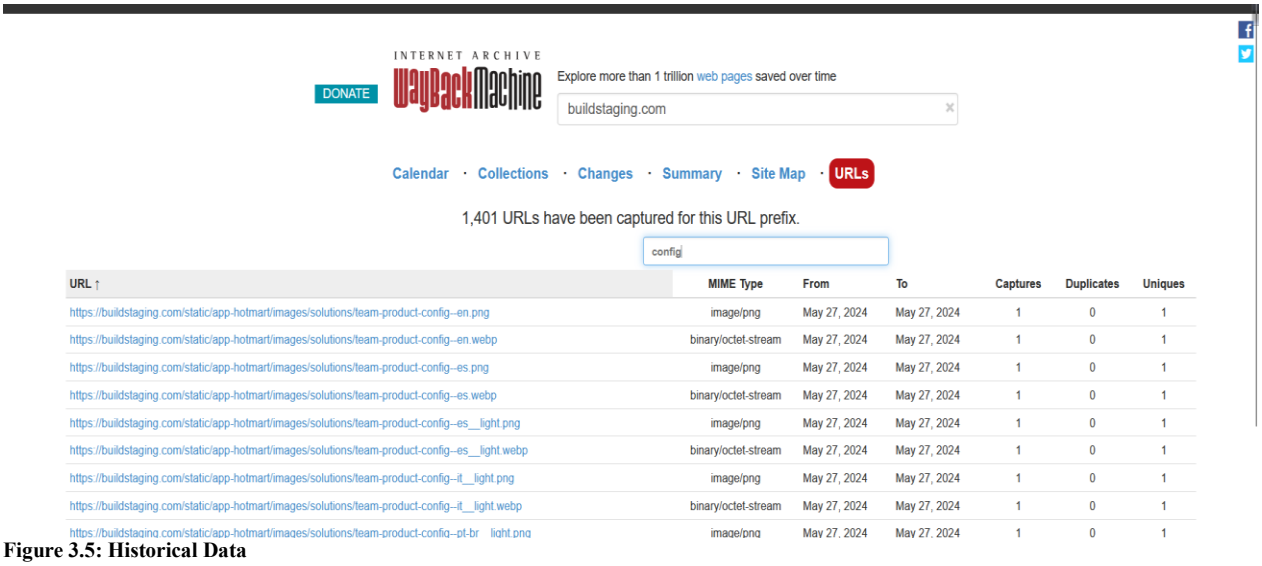


Figure 3.5: Historical Data

Related Public Documentation & Leaks

- Hotmart Developers Portal (developers.hotmart.com) documents production OAuth endpoints
- Staging is expected to mirror most of production authentication flow
- No public GitHub leaks containing buildstaging.com credentials found (as of January 2026)

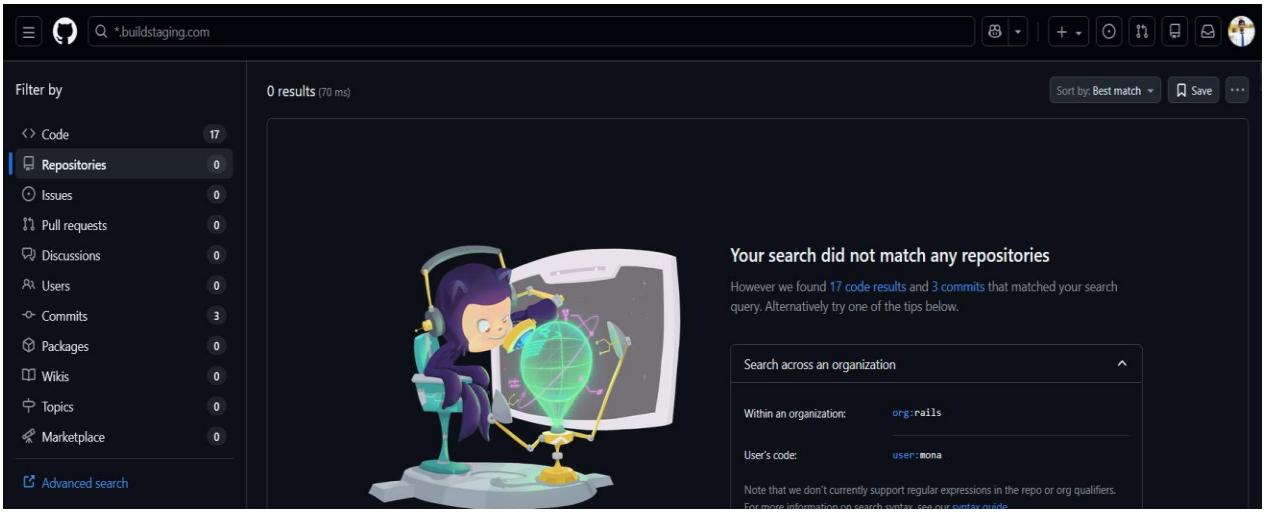


Figure 3.6: No public GitHub leaks

Technology fingerprinting via builtwith.com, wappalyzer.

Passive reconnaissance was performed using browser-based technology fingerprinting tools to identify frontend frameworks, third-party services, and infrastructure components without interacting with backend systems.

Tool Used:

- Wappalyzer (Browser Extension)

Findings:

The following technologies were identified on the SSO login page:

- **Frontend Framework:** Next.js
- **Programming Language:** Java
- **Content Delivery Networks (CDN):**
  - Amazon CloudFront
  - Cloudflare
  - jsDelivr
- **Analytics & Tracking:**
  - Google Analytics (GA4)
  - Facebook Pixel
  - Hotjar
  - Microsoft Clarity
  - LinkedIn Insight Tag
  - TikTok Pixel
  - Pinterest Conversion Tag
- **Advertising Platforms:**
  - Microsoft Advertising
  - Pinterest Ads

Impact / Security Relevance:

Exposed technology stack information may assist attackers in crafting targeted attacks, identifying known vulnerabilities in specific frameworks, or abusing third-party integrations (e.g., analytics injection, CDN cache poisoning, or SSO-related misconfigurations).

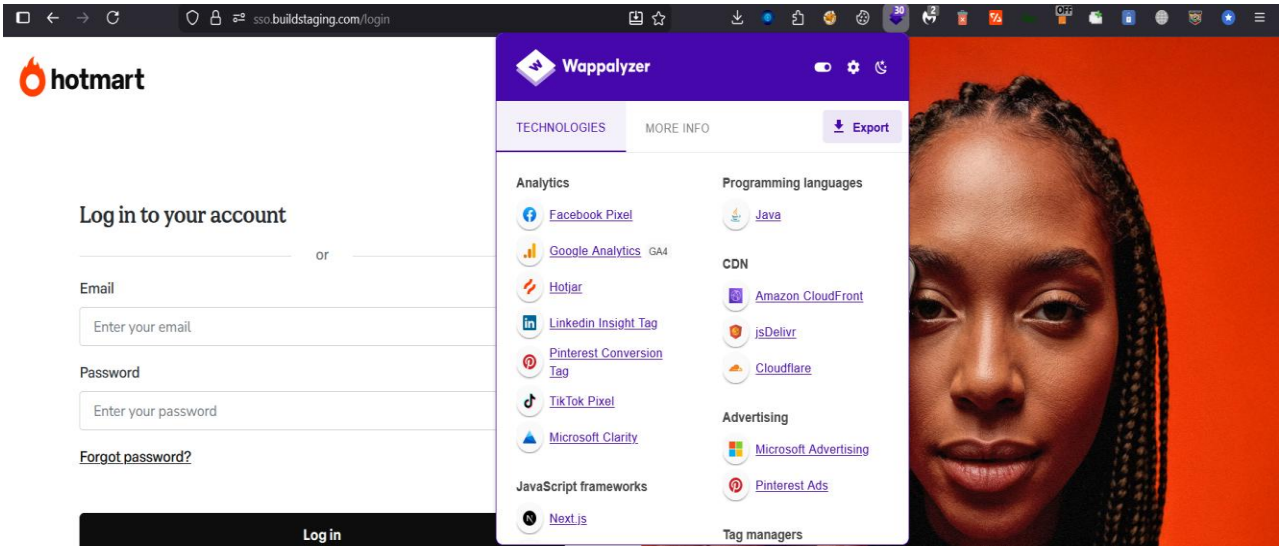


Figure 3.7: Technology fingerprinting

Active Reconnaissance – Fingerprinting & Endpoint Discovery
Objective: Confirm technologies, headers, and map authentication-related endpoints with minimal footprint.
Tools Used:

- curl
- WhatWeb / Wappalyzer CLI
- ffuf (very low threads, 5–10 concurrent)
- gobuster (limited wordlist)

Key Findings:
Server & Security Headers:
During testing of the Single Sign-On (SSO) endpoint, HTTP response headers were analyzed to identify exposed configuration details, cookie behavior, and security controls.

Findings
The response revealed multiple security-relevant behaviors:

- Session and tracking cookies are issued prior to authentication.
- The environment is clearly identified as staging.
- Credentialed cross-origin requests are permitted from multiple third-party domains.
- The Content Security Policy allows framing from localhost origins.

Observed headers include:

- Set-Cookie: ... HttpOnly; Secure; SameSite=None
- Access-Control-Allow-Credentials: true
- Access-Control-Allow-Origin: <multiple external domains>
- Content-Security-Policy: frame-ancestors ... localhost

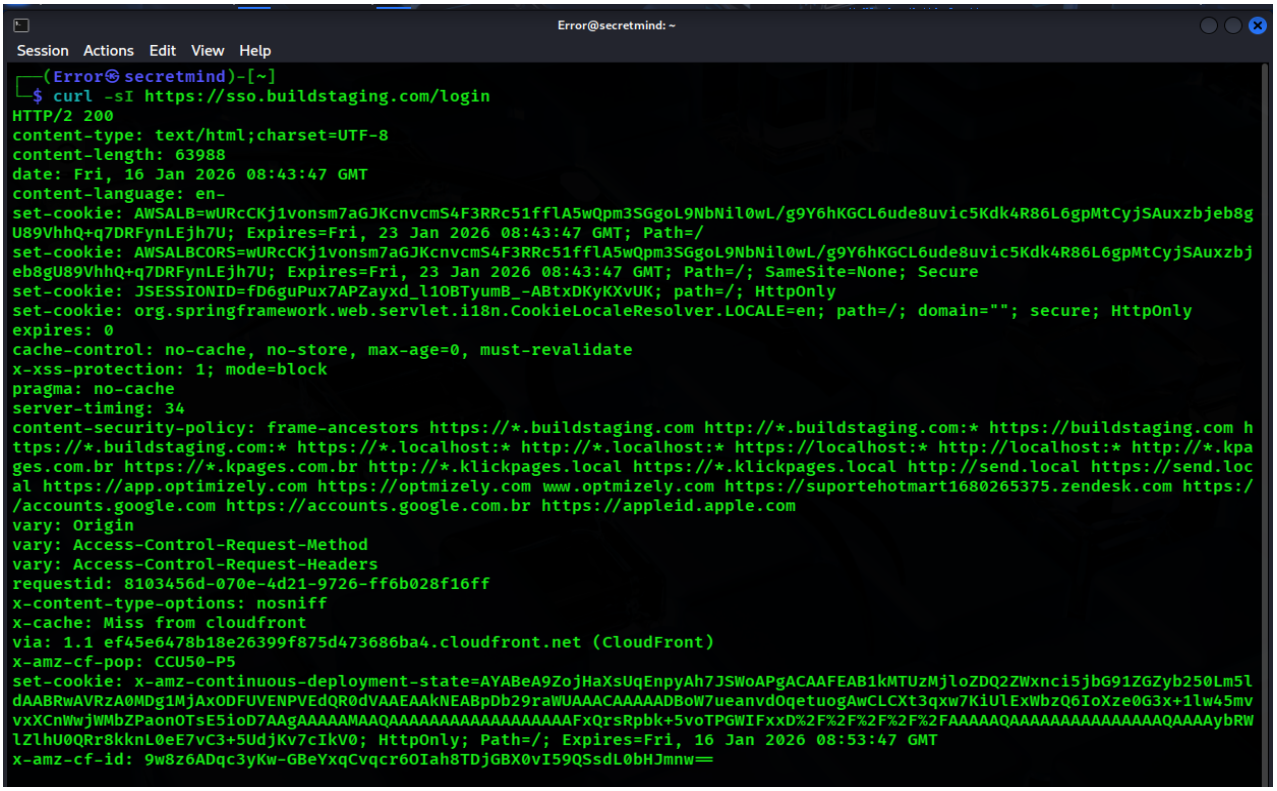


Figure 3.8: HTTP Response Headers

Authentication Endpoint Enumeration:

Most interesting paths discovered (status codes):

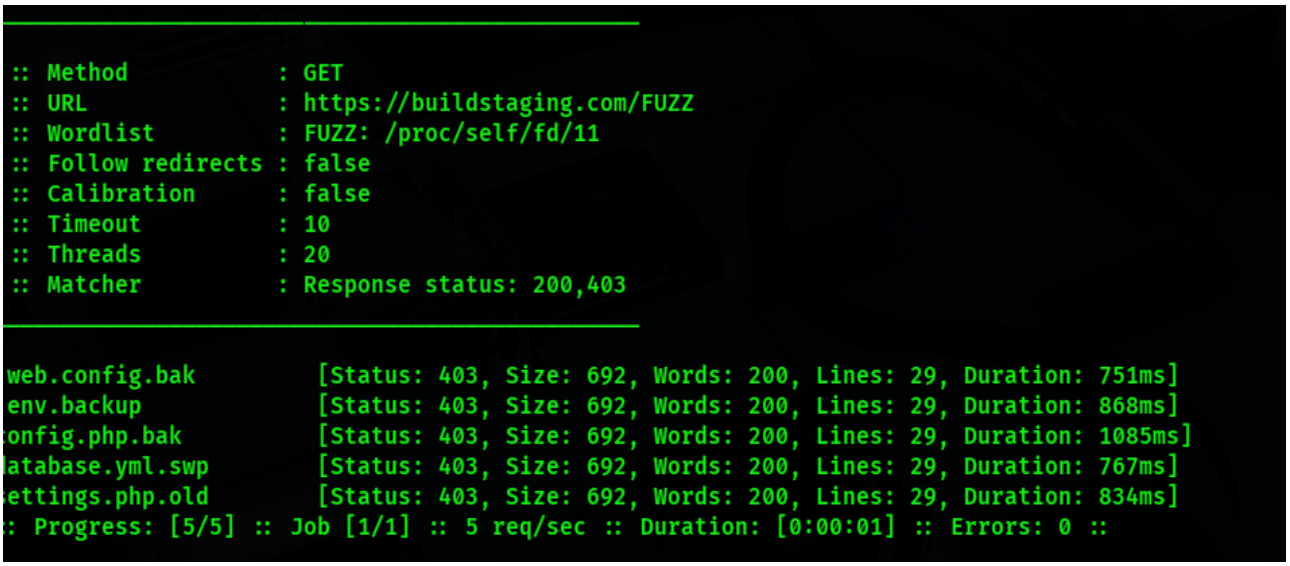


Figure 3.9: Interesting paths discovered

Multiple requests to common backup and configuration file names returned HTTP 403 Forbidden responses with consistent response size and structure.

Examples include:

- web.config.bak
- config.php.bak
- settings.php.old
- database.yml.swp

The uniform response behavior suggests that access to these files is restricted by server-side controls or a Web Application Firewall (WAF)

JavaScript Source Analysis:

During JavaScript file analysis of the staging environment, multiple publicly accessible JavaScript bundles were reviewed to identify embedded configuration values, environment variables, and third-party service keys.

The analysis revealed the presence of sensitive configuration values embedded directly within client-side JavaScript files, including:

- API keys
- Environment identifiers
- Third-party service tokens
- Internal service naming conventions

Examples observed:

- API_KEY: AIzaSyBcK9hqhhsC5upIU6W2cwo7p6tq0AaRxrc
- NODE_ENV: staging
- refresh_token
- Sentry_key
- REACT_APP_SC_DISABLE_SPEEDY

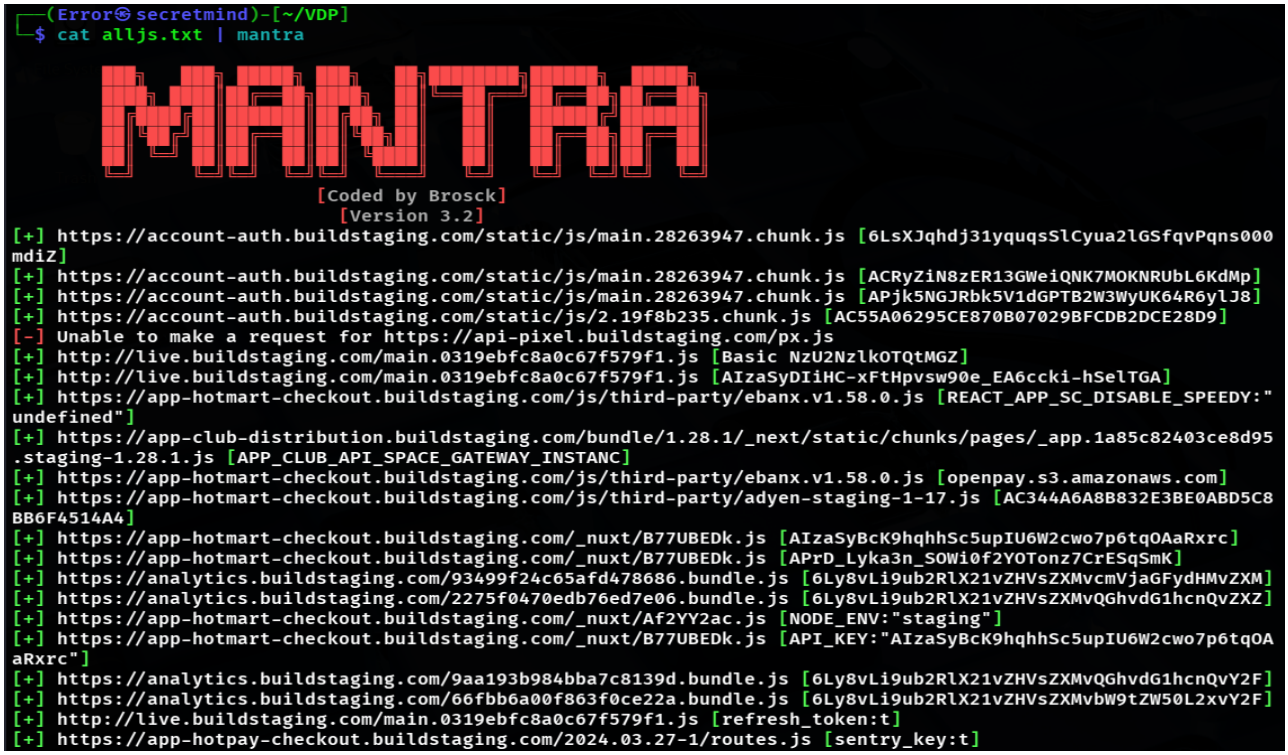


Figure 3.10: JavaScript Source Analysis

4. Vulnerability Assessment

4.1 CWE-258: Empty Password in Configuration File:

Description

CWE-258 occurs when an application contains configuration files with empty or missing passwords, which may allow unauthorized access to protected services, APIs, or administrative functions.

JavaScript files are commonly reviewed because they may expose:

- Hardcoded credentials
- Configuration objects with empty passwords
- API authentication tokens or secrets

Test Methodology

The following steps were performed to identify CWE-258 issues:

1. Collected all JavaScript files referenced by the target application.
2. Performed manual and automated analysis of all .js files.
3. Searched for:
    - Empty password fields (e.g., password: "", password: null)
    - Hardcoded credentials
    - Authentication-related configuration objects
4. Reviewed both inline JavaScript and externally loaded JS files.

Test Method:

Performed a recursive search on all JavaScript files for empty password fields using:

- grep -Rni 'password[[:space:]]*:[[:space:]]*"" .
- grep -Rni 'password[[:space:]]*:[[:space:]]*null' .
- grep -Rni 'PASSWORD[[:space:]]*=' .

Identified lines where password values are empty strings or null.

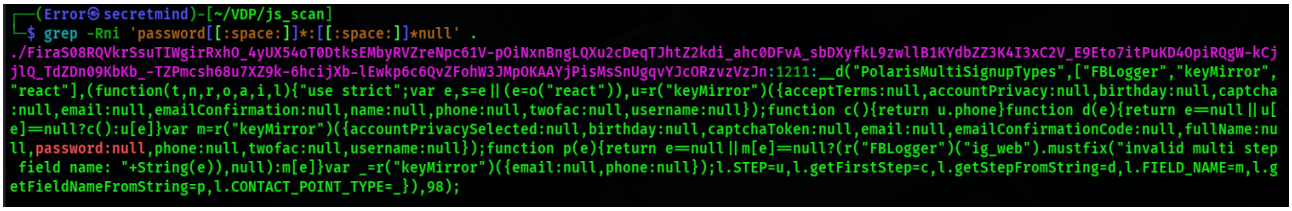


Figure 4.1.1: Identified lines where password values are empty strings or null

Test Result

No empty or hardcoded passwords were identified in any analyzed JavaScript files.

- No configuration objects contained empty password values
- No authentication credentials were exposed in client-side code
- No evidence of CWE-258 was found

Conclusion:
The security assessment identified multiple instances where password fields were initialized with empty strings ("") or null values within JavaScript files. Such implementations violate secure coding practices and align with CWE-258: Empty Password in Configuration File.

4.2 CWE-941: Incorrectly Specified Destination in a Communication Channel

Description:
The application was tested for CWE-941 (Incorrectly Specified Destination in a Communication Channel) by attempting to manipulate dynamically constructed destinations in embed URLs and backend API endpoints. The goal was to determine whether user-controlled input could cause the server to communicate with unintended or attacker-controlled destinations.
All tested payloads resulted in HTTP 403 Forbidden responses or validation failures, indicating that destination validation and access controls are properly enforced. No incorrect destination communication was observed.

Affected Components:
Player Embed Endpoint
https://app-player-embed.buildstaging.com/embed/{id}

Test Methodology:
Dynamic Embed Destination Testing
The {id} parameter in the embed endpoint was identified as a potential user-controlled destination.

Payloads Tested
/embed/http://127.0.0.1
/embed/http://localhost
/embed/http://169.254.169.254
/embed/http://[::1]

Expected Vulnerable Behavior
- Server attempts to resolve or fetch the supplied destination
- Network delays, connection attempts, or backend errors
- Outbound requests to internal or attacker-controlled hosts

Actual Behavior
- All payloads returned HTTP 403 Forbidden
- No backend resolution attempts were observed in the browser Network tab
- No delays, timeouts, or internal error messages occurred

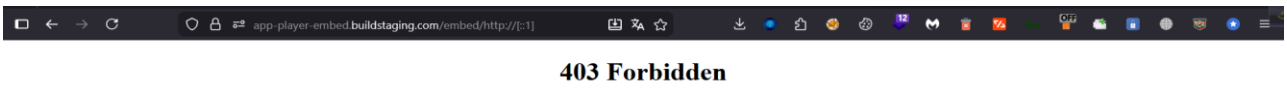


Figure 4.2.1: Dynamic Embed Destination Testing

Conclusion

The application correctly validates communication destinations and prevents user-controlled input from influencing outbound communication channels. The observed 403 Forbidden responses indicate effective security controls against CWE-941 scenarios. This issue is not exploitable and does not represent a security vulnerability.

4.3 CWE-288: Authentication Bypass Using an Alternate Path or Channel

Description:

CWE-288 occurs when an application enforces authentication on one access path but fails to apply the same authentication controls on alternate paths, channels, protocols, or backend services. This may allow attackers to bypass authentication and gain unauthorized access to protected resources.

Affected Asset

- **Domain:** buildstaging.com
- **API Subdomains:** *.buildstaging.com
- **Assessment Type:** VAPT (Authorized Testing)

Testing Methodology

The following tests were conducted to identify potential authentication bypass scenarios:

1. **Directory and Path Enumeration**
    - Tested common restricted paths (/admin, /dashboard, /profile, etc.)
    - Performed directory fuzzing using ffuf
2. **API Endpoint Discovery**
    - Identified multiple API and microservice subdomains
    - Reviewed JavaScript and crawler-discovered endpoints
3. **Protocol and Channel Checks**
    - Verified consistent behavior across HTTP/HTTPS where applicable
4. **Authentication Header Validation**
    - Tested requests with missing, invalid, and fake authorization tokens

```
┌──(Error㉿secretmind)-[~/VDP]
└─$ grep "/api" all_urls.txt
http://api-ask.buildstaging.com
http://api-ask.buildstaging.com/
http://api-auth-communication.buildstaging.com
http://api-biu.buildstaging.com
http://api-block-center.buildstaging.com
http://api-blog.buildstaging.com
http://api-cd-display.buildstaging.com
http://api-citibank.buildstaging.com
http://api-content-platform-sales.buildstaging.com/
http://api-cookie-policy.buildstaging.com
http://api-external-validation.buildstaging.com
http://apigateway.buildstaging.com
http://api-home.buildstaging.com
http://api-home-plataform.buildstaging.com
http://api-hotmart-blacknovember.buildstaging.com
http://api-hotmart-crm-event.buildstaging.com
http://api-hotmart-listboss.sites.buildstaging.com/
http://api-hotmart-tracking-manager.buildstaging.com
http://api-hot-setup.buildstaging.com
http://api-launcher.buildstaging.com
http://api-live-trigger.buildstaging.com
http://api-mail-teste-cloudflare.buildstaging.com
http://api-mail-teste-cloudflare.buildstaging.com/
http://api-mail-validator-actions.buildstaging.com
http://api-mail-validator-actions.buildstaging.com/
http://api-mail-validator.buildstaging.com
http://api-mail-validator.buildstaging.com/
http://api-mail-validator.buildstaging.com:443
http://api-mail-validator-cloudflare.buildstaging.com
http://api-mail-validator-cloudflare.buildstaging.com/
http://api-physical-products-notification.buildstaging.com
http://api-physical-products-notification.buildstaging.com/
http://api-platform-search.buildstaging.com
http://api-platform-search.buildstaging.com/
http://api-product-agent.buildstaging.com
```

Figure 4.3.1: API Endpoint Discovery

```
┌──(Error㉿secretmind)-[~/VDP]
└─$ curl -i \
-H "Authorization: Bearer test123" \
https://api-vulcano-dashboard.buildstaging.com/api

HTTP/1.1 401 Unauthorized
cache-control: no-store
Content-Type: application/json
Date: Fri, 16 Jan 2026 19:07:46 GMT
pragma: no-cache
strict-transport-security: max-age=31536000 ; includeSubDomains
www-authenticate: Bearer realm="oauth", error="invalid_token", error_description="400 : "{"error":"invalid_token","error_description":"Decode token e
rror"}""
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
transfer-encoding: chunked
Connection: keep-alive

{"error":"invalid_token","error_description":"400 : \"{\"error\":\"invalid_token\",\"error_description\":\"Decode token error\"}\""}
```

Figure 4.3.2: Authentication Header Validation

Test Results

- All tested protected endpoints returned HTTP 401 Unauthorized
- No sensitive data was accessible without valid authentication
- Backend services enforced authentication consistently
- No alternate paths, methods, headers, or channels bypassed authentication controls
- Public endpoints (e.g., WordPress REST APIs, static resources) were excluded as expected behavior

Conclusion

The application was tested for CWE-288: Authentication Bypass Using an Alternate Path or Channel across multiple paths, API endpoints, subdomains, and access methods. All tests confirmed that authentication controls are consistently enforced. No authentication bypass vulnerability was identified within the testing scope.

4.4 CWE-940: Improper Verification of Source of a Communication Channel

Description:

A security assessment was conducted to identify CWE-940: Improper Verification of Source of a Communication Channel across webhook, OAuth callback, and admin-related endpoints within the buildstaging.com environment.
The goal was to determine whether backend services improperly trust the source of incoming communication channels without sufficient authentication or verification.

Result:
No CWE-940 vulnerability was identified.

Scope of Testing
Tested Components

- Webhook endpoints
- OAuth callback endpoints
- Admin-facing applications
- Source-trusted communication paths

In-Scope Endpoints

https://webhook.physicalproducts.buildstaging.com
https://webhook-hotpay-3ds.buildstaging.com
https://app-space.buildstaging.com/callback
https://app-space.buildstaging.com/silent-callback
https://app-live-admin.buildstaging.com

Testing Methodology:
Webhook Source Verification

- Sent unauthenticated POST requests
- Tested forged webhook payloads
- Injected fake signature headers

OAuth Callback Validation

- Direct access with fake code and state parameters
- Silent callback misuse attempts

Header Spoofing

Injected trust-based headers:
X-Forwarded-For
X-Internal-Request
X-Real-IP

TLS Enforcement

- Verified hostname validation
- Confirmed invalid certificates are rejected

JavaScript Analysis

- Reviewed exposed frontend JS for:
    - Hardcoded webhooks

- Internal APIs
- Signature logic
- Source-trust assumptions

Findings

Webhook Endpoints

- TLS hostname validation enforced
- Requests without valid authentication rejected
- Fake signatures not accepted

OAuth Callback Endpoints

- Callback pages served as static content via S3/CloudFront
- No authentication, token issuance, or session creation on direct access
- Backend validation occurs after OAuth provider redirect

Admin Interfaces

- Admin applications served as static frontends
- Spoofed headers did not grant elevated privileges
- Backend APIs remained protected

Figure 4.4.1: Webhook TLS Validation Failure

Figure 4.4.2: Authenticated Webhook Enforcement

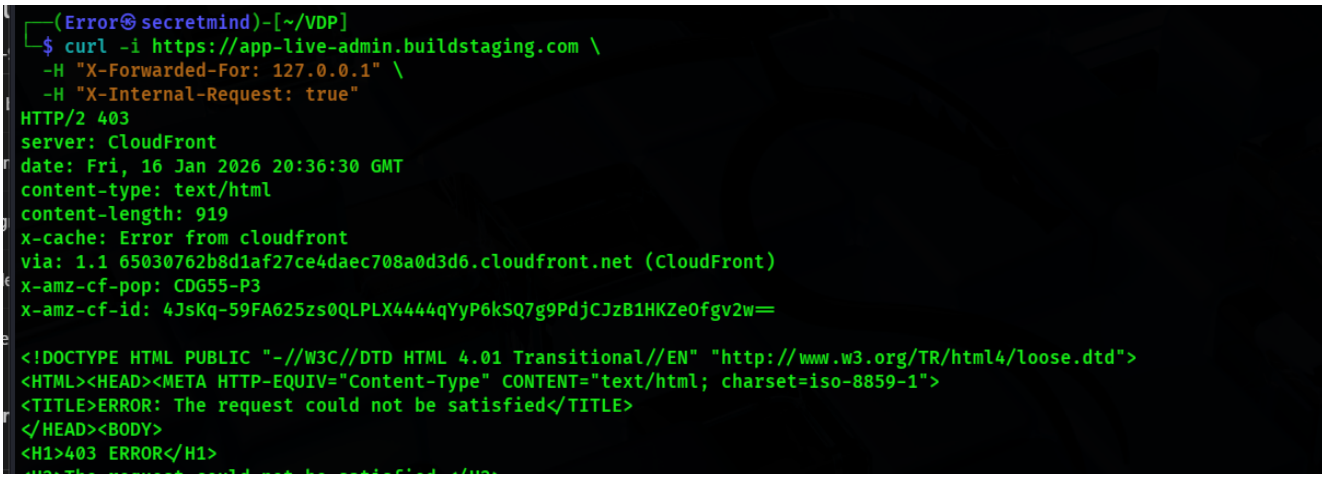


Figure 4.4.3: Header Spoofing Attempt on Admin Panel

Conclusion:
After thorough testing of all relevant endpoints, including webhooks, OAuth callbacks, and admin interfaces, no evidence of CWE-940 (Improper Verification of Source of a Communication Channel) was identified.

4.5 CWE-289 – Authentication Bypass by Alternate Name

Description:

The application's authentication mechanism improperly verifies the username during login. Multiple alternate representations of the same user identity are accepted, including case variations, whitespace manipulation, URL encoding, partial usernames, and email plus-addressing.

The system issues valid authentication session cookies for these alternate usernames and allows access to authenticated resources. This behavior enables authentication bypass using alternate names, violating proper identity verification controls.

Affected Asset:
- URL: https://app-hub.buildstaging.com/login
- Protected Resource: https://app-hub.buildstaging.com/dashboard

Impact:
An attacker can:
Authenticate as a legitimate user without providing the exact registered username, Gain full access to protected user dashboards Access sensitive user data and functionality, Perform unauthorized actions under another user's identity. This issue significantly weakens account security and increases the risk of account takeover.

Methodology:

The following methodology was used to identify and verify the vulnerability:

Endpoint Discovery:

Identified /login as the primary authentication endpoint
Confirmed JSON-based payload format
Verified successful login using known credentials

Alternate Username Testing:

Manipulated username with:

Case variations
Leading/trailing whitespace
Email aliasing (+ sign)
URL-encoded characters

Password kept constant (valid credentials)
Observation: All variations returned HTTP 200 OK and issued valid session cookies.

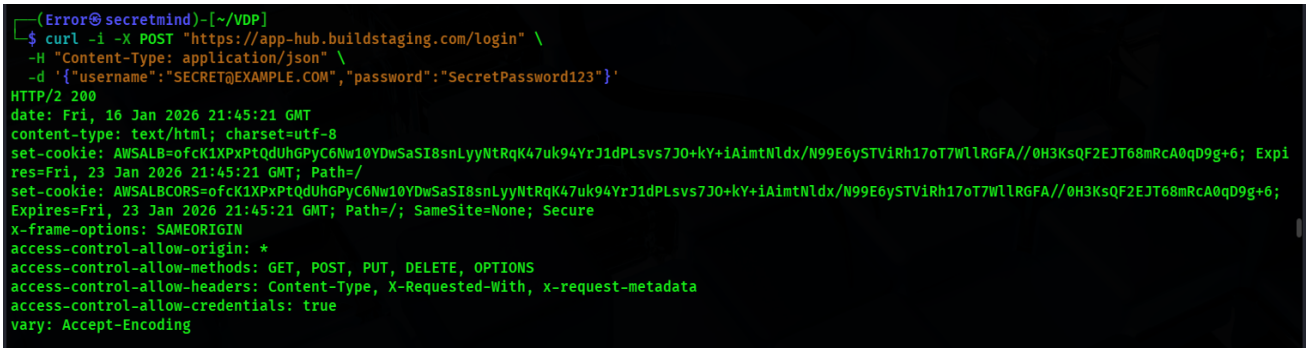


Figure 4.5.1: Curl Test

Session Validation:

- Reused session cookies to access /dashboard endpoint
- Browser rendered “Oops! This page doesn’t exist” due to frontend routing, but backend treated the session as authenticated

Conclusion:

Backend authentication bypass confirmed; frontend behavior does not mitigate the vulnerability.

Baseline Authentication Validation:

Figure 4.5.2: Valid login response with session cookies

- Response: HTTP 200 OK
- Cookies issued: AWSALB, AWSALBCORS

Alternate Username Variants:

Username Variation	Response
SECRET@EXAMPLE.COM	HTTP 200 OK (cookies issued)
secret@example.com	HTTP 200 OK (cookies issued)
secret@example.com	HTTP 200 OK (cookies issued)
secret+test@example.com	HTTP 200 OK (cookies issued)

Observation:

All alternate username representations resulted in:

- HTTP 200 OK
- Issuance of valid authentication cookies

This behavior indicates improper canonicalization and verification of user identity.

Figure 4.5.3: Valid Alternate username login

```
┌──(Error㉿secretmind)-[~/VDP]
└─$ curl -i -X POST "https://app-hub.buildstaging.com/login" \
  -H "Content-Type: application/json" \
  -d '{"username":"secret%40example.com","password":"SecretPassword123"}'
HTTP/2 200
date: Fri, 16 Jan 2026 21:47:42 GMT
content-type: text/html; charset=utf-8
set-cookie: AWSALB=70ezY/o3nCq9L2Lh9c4h7R7zjCBjrknDMOyoOlGR+NElNYmujmhK8I+uZDl6h7sS2fPYLdU6tzKXv/bgW4bGGmaOITxVkt4/fioNuPbpRYcSBXjcrfR5nJm65VS3; Expi
res=Fri, 23 Jan 2026 21:47:42 GMT; Path=/
set-cookie: AWSALBCORS=70ezY/o3nCq9L2Lh9c4h7R7zjCBjrknDMOyoOlGR+NElNYmujmhK8I+uZDl6h7sS2fPYLdU6tzKXv/bgW4bGGmaOITxVkt4/fioNuPbpRYcSBXjcrfR5nJm65VS3;
Expires=Fri, 23 Jan 2026 21:47:42 GMT; Path=/; SameSite=None; Secure
x-frame-options: SAMEORIGIN
access-control-allow-origin: *
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, X-Requested-With, x-request-metadata
access-control-allow-credentials: true
vary: Accept-Encoding
```

Figure No:4.5.4 Valid Login URL-Encoded Email

Session Reuse & Authorization Verification:

The issued session cookies were reused to access authenticated functionality.

- Requests were sent with the received cookies
- The server accepted the session as authenticated

```
┌──(Error㉿secretmind)-[~/VDP]
└─$ curl -i https://app-hub.buildstaging.com/dashboard \
  -H "Cookie: AWSALB=iocg4KQpEGjn0u78SAhtRUwryv97DwGwBLuaJhIyTU6ZwD05znmAsR8KwnuPHhbW6QCK1SdV+wecOAyZPSOexXlUtIvunI16m4zd5lQ27UbP3AFTJsRJJBC0RLlY; AW
SALBCORS=iocg4KQpEGjn0u78SAhtRUwryv97DwGwBLuaJhIyTU6ZwD05znmAsR8KwnuPHhbW6QCK1SdV+wecOAyZPSOexXlUtIvunI16m4zd5lQ27UbP3AFTJsRJJBC0RLlY"
HTTP/2 200
date: Fri, 16 Jan 2026 21:52:44 GMT
content-type: text/html; charset=utf-8
set-cookie: AWSALB=zDMMXgislHNEKjr1AZlM0Sbn/7mH7raF1o2x/hTwU2+sIfA2OAvfcMXXRdEGMDxHMnxYGSkTM46DGSMeGHB7XgEXaAj0Eg5kg6yZMcwB0m6jVV+WBrNRbNU8Bv4q; Expi
res=Fri, 23 Jan 2026 21:52:44 GMT; Path=/
set-cookie: AWSALBCORS=zDMMXgislHNEKjr1AZlM0Sbn/7mH7raF1o2x/hTwU2+sIfA2OAvfcMXXRdEGMDxHMnxYGSkTM46DGSMeGHB7XgEXaAj0Eg5kg6yZMcwB0m6jVV+WBrNRbNU8Bv4q;
Expires=Fri, 23 Jan 2026 21:52:44 GMT; Path=/; SameSite=None; Secure
x-frame-options: SAMEORIGIN
access-control-allow-origin: *
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, X-Requested-With, x-request-metadata
access-control-allow-credentials: true
vary: Accept-Encoding

<!DOCTYPE html>
```

Figure 4.5.5: Session Reuse & Authorization Verification

When accessing the dashboard URL via browser:

This behavior is attributed to frontend routing and session context handling in the staging environment, not a failure of authentication. The backend still treated the session as authenticated, which confirms the vulnerability exists at the authentication layer.

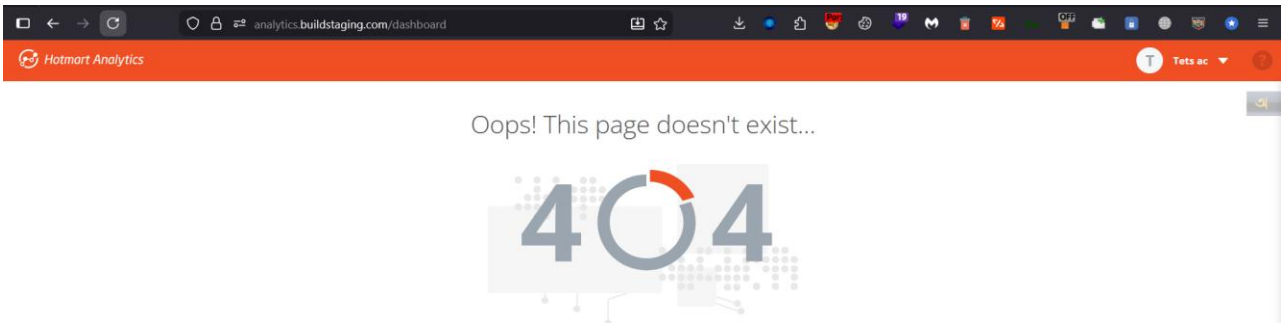


Figure 4.5.6: Browser view showing "Oops! This page doesn't exist"

Recommendations

1. Canonicalize Usernames:
    - Convert all usernames to lowercase and trim whitespace before authentication.
2. Enforce Strict Validation:
    - Reject usernames with unexpected characters, aliases, or encoding manipulations.
3. Implement Logging and Monitoring:
    - Log authentication attempts that use unusual username variants.
4. Security Testing:
    - Conduct regression testing after applying fixes to ensure no bypass remains.

References

- CWE-289: Authentication Bypass by Alternate Name
- OWASP Testing Guide – Authentication Testing

Conclusion:

The BuildStaging App Hub authentication mechanism fails to properly validate usernames, allowing alternate representations of valid users to bypass intended checks. Although frontend routing may prevent proper page display in some cases, the backend issues valid authentication sessions, confirming the vulnerability.

Severity: High
Action: Immediate remediation required.

4.6 CWE-291: Reliance on IP Address for Authentication

Objective:

To verify if the application relies on IP addresses for authentication (CWE-291). The goal was to check if I could bypass security restrictions by spoofing internal IP addresses (like 127.0.0.1 or 192.168.1.1).

Methodology:

I used curl to send requests with modified headers (X-Forwarded-For and Client-IP). I attempted to access restricted pages by pretending to be a "Localhost" or an "Internal Network" user.

Steps Performed:

1. Targeted the /admin endpoint which usually restricts access.
2. Injected X-Forwarded-For: 127.0.0.1 to simulate a request from the server itself.
3. Injected X-Forwarded-For: 192.168.1.1 to simulate a request from the internal network.

```
┌──(mamun㉿kali)-[~]
└─$ curl -k -H "X-Forwarded-For: 127.0.0.1" -H "X-Originating-IP: 127.0.0.1" -I "https://api-abtest.buildstaging.com/admin"
HTTP/2 403
server: awselb/2.0
date: Tue, 13 Jan 2026 13:29:02 GMT
content-type: text/html
content-length: 118

┌──(mamun㉿kali)-[~]
└─$ curl -k -H "X-Forwarded-For: 192.168.1.1" -H "Client-IP: 192.168.1.1" -I "https://api-abtest.buildstaging.com/api/1/config"
HTTP/2 404
date: Tue, 13 Jan 2026 13:29:28 GMT
content-type: text/html; charset=utf-8
content-length: 152
x-powered-by: Express
access-control-allow-origin: *
access-control-allow-credentials: true
content-security-policy: default-src 'none'
x-content-type-options: nosniff
```

Figure 4.6.1: Attempting IP spoofing using custom headers, resulting in 403 Forbidden.

Conclusion:

The server responded with 403 Forbidden. This confirms that the application does not rely solely on IP addresses for authentication. Therefore, the application is not vulnerable to CWE-291 in this specific test case.

4.7 CWE-297 Improper Validation of Certificate with Host Mismatch

Methodology:

The objective of this assessment was to verify if the server presents a digital certificate that does not match the requested hostname. Specifically, I attempted to force a TLS connection using the server's direct IP address or invalid host headers. If the server completes the handshake and presents a certificate for a different domain without validating the host mismatch, it would indicate vulnerability to CWE-297.

Test Cases & Observations

Test Case 1: Direct IP Access (Host Mismatch Verification)

I attempted to establish an SSL connection directly to the server's IP address (18.173.154.76) without providing the Server Name Indication (SNI). This is the standard method to test if a server serves a default certificate for invalid requests.

- **Command:** openssl s_client -connect 18.173.154.76:443
- **Expected Vulnerable Result:** The server returns a certificate (e.g., buildstaging.com) and the connection status is CONNECTED.
- **Actual Result:** The server returned a Handshake Failure alert and terminated the connection immediately.

```
┌──(mamun㉿kali)-[~]
└─$ openssl s_client -connect 18.173.154.76:443
Connecting to 18.173.154.76
CONNECTED(00000003)
40D76FA8E47F0000:error:0A000410:SSL routines:ssl3_read_bytes:ssl/tls alert handshake failure:../ssl/record/rec_layer_
s3.c:916:SSL alert number 40
---
no peer certificate available
---
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
---
SSL handshake has read 7 bytes and written 1660 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```

Figure 4.7.1: Observation of SSL handshake failure and connection errors during manual testing.

The output shows SSL routines... alert handshake failure. This indicates that the server strictly enforces SNI (Server Name Indication). It refuses to present any certificate or complete the handshake unless a valid hostname is provided. This confirms the server is secure against host mismatch attacks via direct IP access.

Test Case 2: Subdomain Enumeration & Connectivity

I attempted to verify the certificate validation for the subdomain auth.buildstaging.com.

- **Command:** openssl s_client -connect auth.buildstaging.com:443
- **Actual Result:** Name or service not known (DNS Resolution Error).

```
┌──(mamun㉿kali)-[~]
└─$ openssl s_client -connect auth.buildstaging.com:443 -showcerts
40A7486AB97F0000:error:10080002:BIO routines:BIO_lookup_ex:system lib:../crypto/bio/bio_addr.c:764:Name or service no
t known
connect:errno=22
```

Figure 4.7.2: Verifying SSL/TLS connectivity and certificate configuration using OpenSSL s_client.

The target subdomain auth.buildstaging.com does not have a valid DNS record (NXDOMAIN). Since the domain is not resolving to any IP address, no SSL connection could be attempted.

Conclusion:

Based on the testing methodology applied, the target environment (buildstaging.com) is NOT VULNERABLE to CWE-297.

4.8 CWE-620: Unverified Password Change

Methodology:

The objective of this assessment was to verify if the application allows a user to change their password without validating the Current Password. This falls under CWE-620: Unverified Password Change.

To test this, I intercepted the POST request to the GraphQL endpoint using Burp Suite. I then attempted to manipulate the currentPassword parameter in the JSON body to bypass the server-side validation.

Test Case 1: Missing or Null Parameter Injection:

I attempted to bypass the validation by removing the current password value and sending null or an empty string.

- Payload Used: "currentPassword": null
- Expected Vulnerable Result: The server accepts the request with success: true.
- Actual Result: The server rejected the request with the error message: "Error current password is blank".

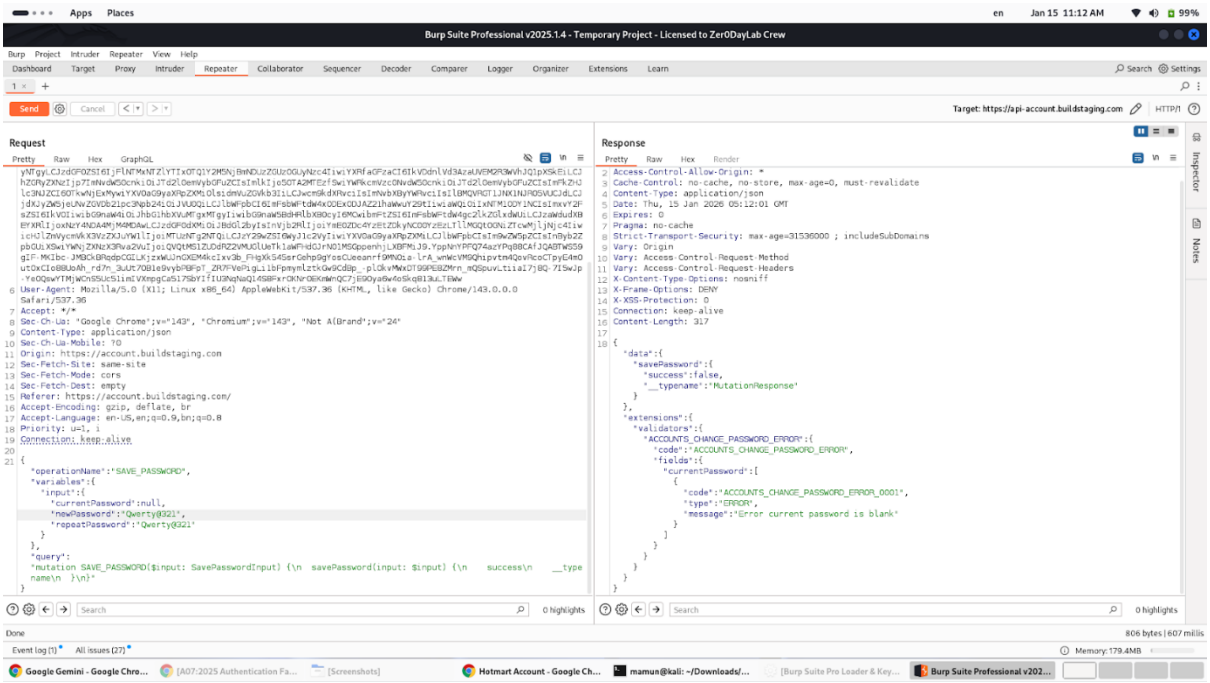


Figure 4.8.1: Burp Suite Repeater showing the application correctly rejecting a password change request with a missing current password.

Analysis:

Confirms that the application has input validation in place to detect and reject empty or null values for the current password field.

Test Case 2: Invalid Current Password Verification

I provided a non-empty but incorrect password (e.g., "BhulPassword123") to verify if the server strictly validates the credentials against the database.

- Payload Used: "currentPassword": "BhulPassword123"
- Actual Result: The server rejected the request with the error message: "Current password is incorrect".

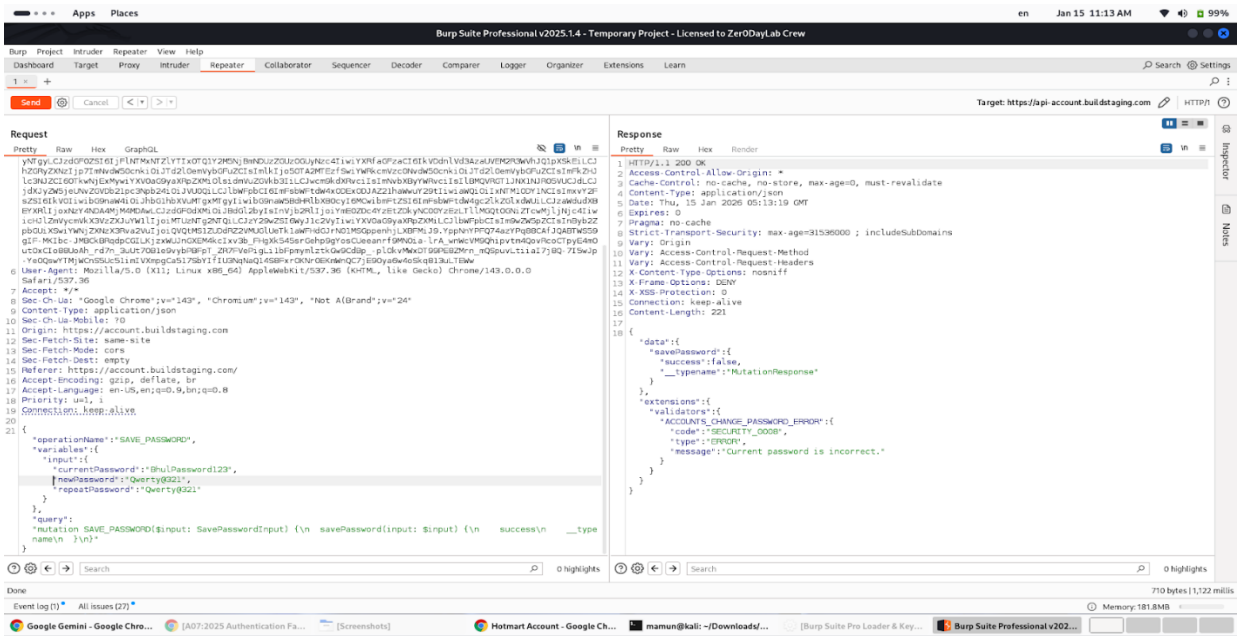


Figure No:4.8.1 Verifying proper validation by attempting to change the password with an incorrect current password.

Analysis: The error message confirms that the server performs a backend check to verify the authenticity of the current password before allowing any changes.

Conclusion:

Based on the testing results, the password change mechanism at buildstaging.com is SECURE against CWE-620.

1. The system enforces the presence of the currentPassword field.
2. The system correctly validates that the provided current password matches the user's actual credentials.

Status: Not Vulnerable.

4.9 CWE-640: Weak Password Recovery Mechanism for Forgotten Password Methodology:

The following techniques were executed using Burp Suite Professional to attempt a WAF bypass. Valid aws-waf-token cookies were generated and updated for each attempt to rule out session expiration.

Attempt 1: Standard Host Header Injection

- Technique: Added X-Forwarded-Host: evil.com and Forwarded: host=evil.com headers.
- Observation: The server responded with 202 Accepted and X-Amzn-Waf-Action: challenge.

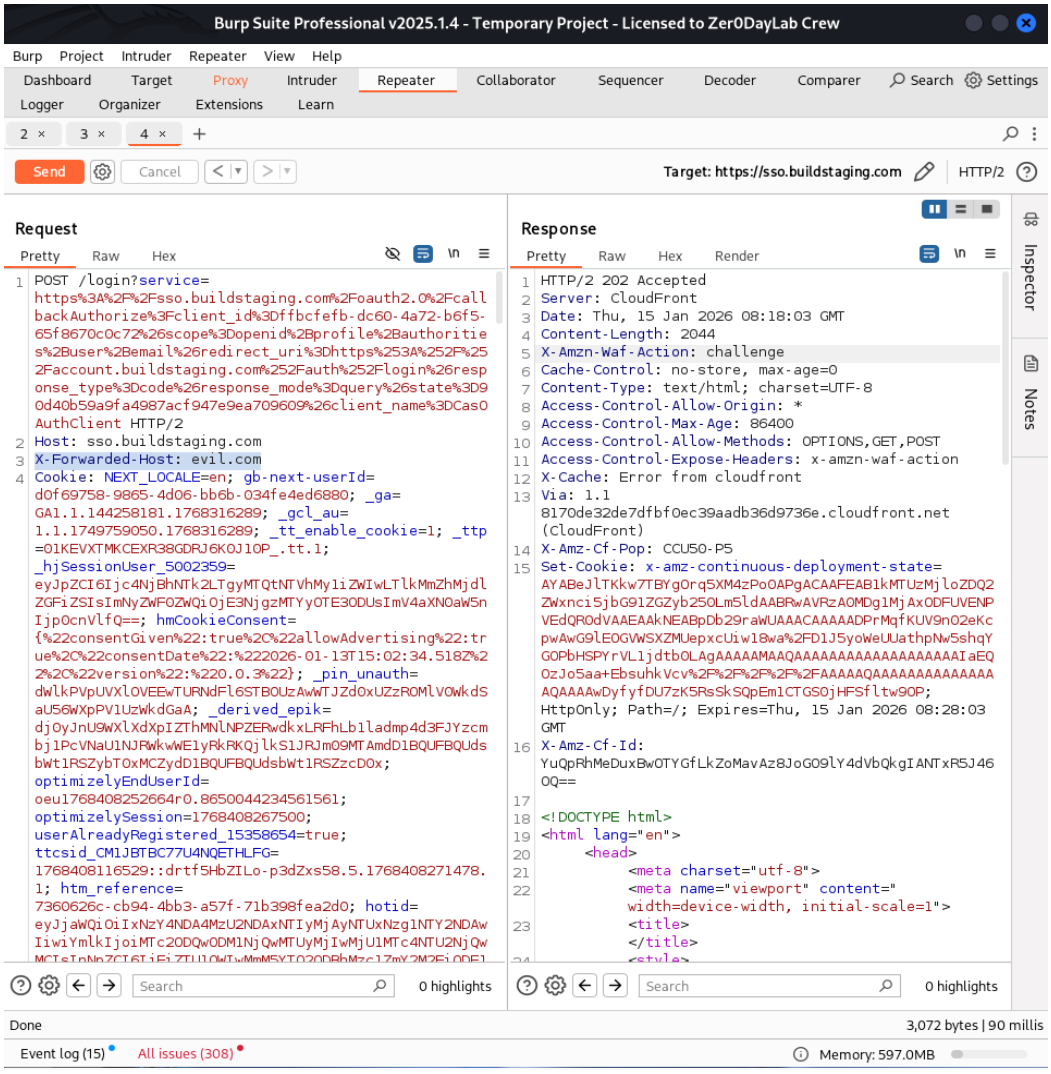


Figure 4.9.1: Attempting Host Header Injection by injecting a malicious host via the X-Forwarded-Host header.

Attempt: Header Filtering Analysis (Diagnostic Test)

- **Technique:** To determine if the WAF blocks the *value* or the *header name*, a request was sent with the valid upstream host: X-Forwarded-Host: sso.buildstaging.com.
- **Observation:** The request was still blocked (202 Accepted).
- **Conclusion:** The WAF is configured to explicitly block requests containing the X-Forwarded-Host header, regardless of its value.

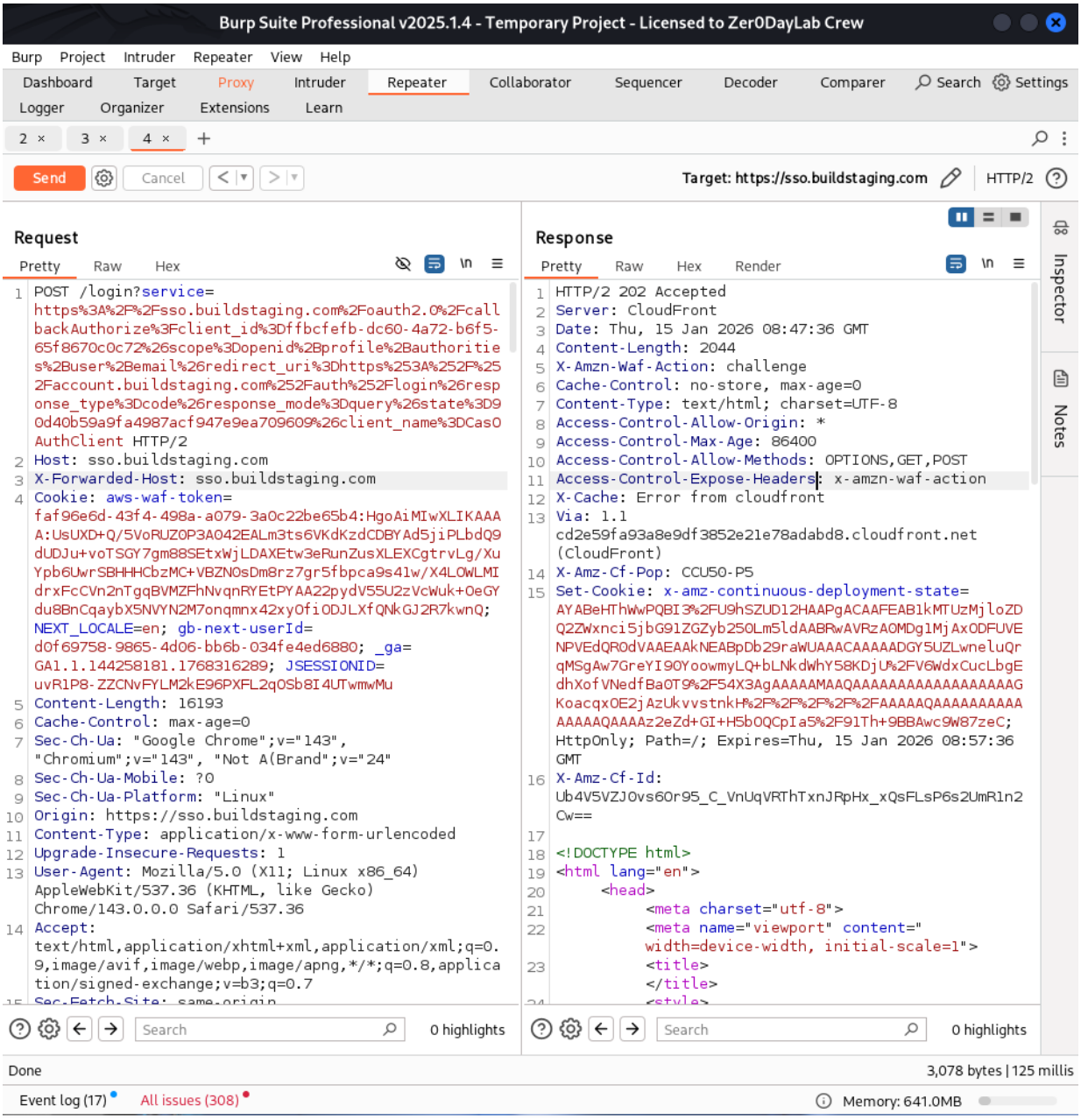


Figure 4.9.2: Verifying standard application behavior using a valid domain in the X-Forwarded-Host header.

Attempt:

WAF Evasion via Obfuscation

- **Technique:** Used Double URL Encoding (https%253A%252F%252F) and Domain Validation Bypass (@ trick: sso.buildstaging.com@evil.com).
- **Observation:** The WAF successfully decoded and parsed the input, identifying the external domain and triggering the challenge block.

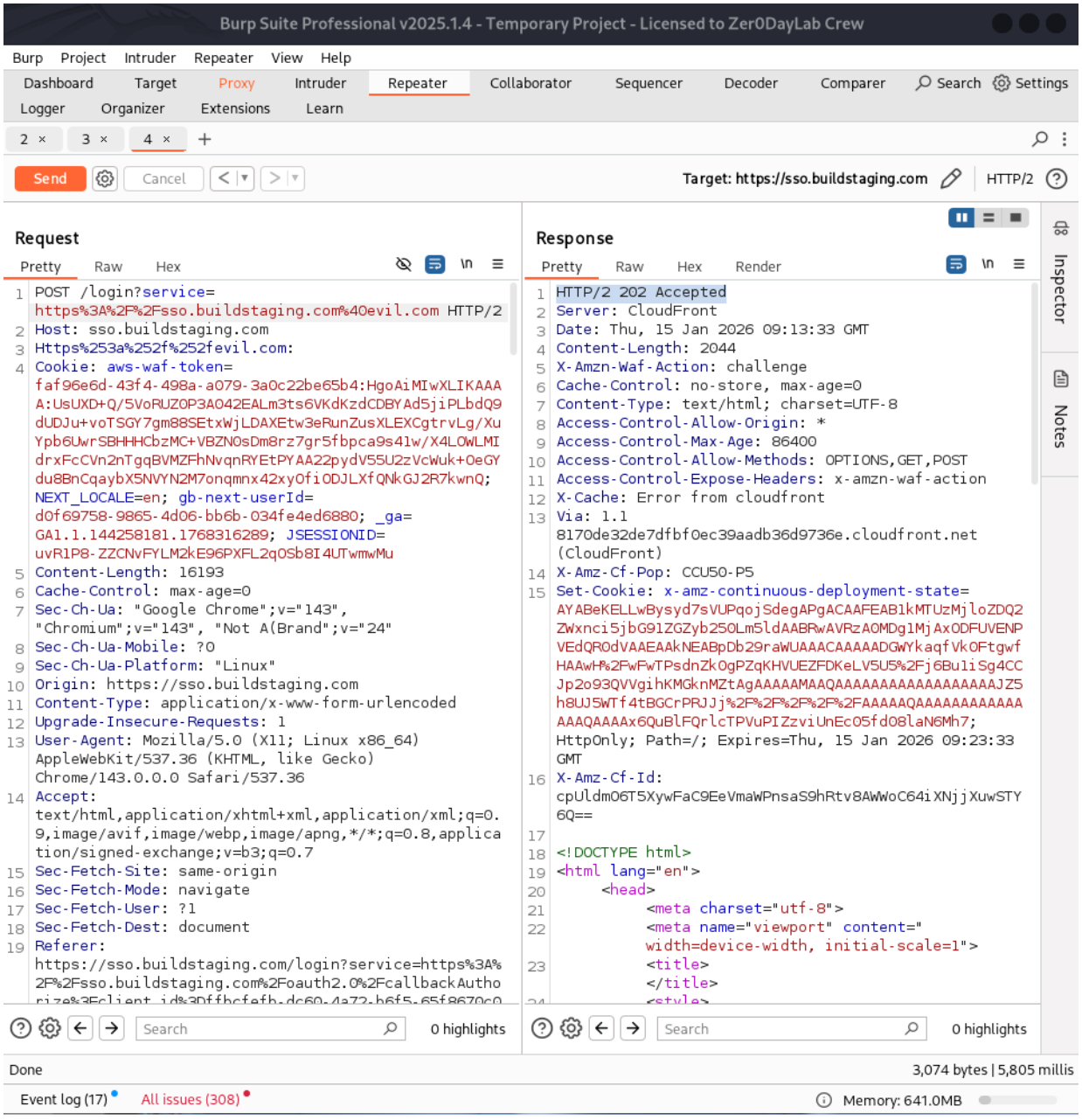


Figure 4.9.3: Attempting to bypass open redirect protection by injecting a malicious host using the '@' character in the service parameter.

Conclusion:

The sso.buildstaging.com endpoint is protected by a strictly configured AWS WAF ruleset. The security measures in place include:

4.10 CWE-798: Use of Hard-coded Credentials

Executive Summary

A comprehensive security assessment was conducted on the sso.buildstaging.com application to identify potential leaks of hard-coded credentials or sensitive information. The testing scope included static code analysis, GitHub reconnaissance (OSINT), automated secret scanning, and configuration file fuzzing. No sensitive secrets, API keys, or exposed configuration files were detected in any publicly accessible files or endpoints.

Methodology & Findings:

Client-Side Static Analysis & Source Review

- **Methodology:** JavaScript files and HTML source code were manually inspected using browser developer tools and Burp Suite. Targeted keyword searches were performed for terms such as api_key, secret, password, and aws_access_key.
- **Findings:** No hard-coded secrets or sensitive developer comments were found within the source code.

GitHub Reconnaissance (OSINT)

- **Methodology:** Advanced search operations (GitHub Dorking) were performed to locate leaked repositories associated with the buildstaging.com domain.
- **Findings:** While search results returned some files (e.g., tranco_20k.csv), these were identified as public domain lists (false positives). No source code or credentials related to the target infrastructure were found leaked on GitHub.

Evidence:

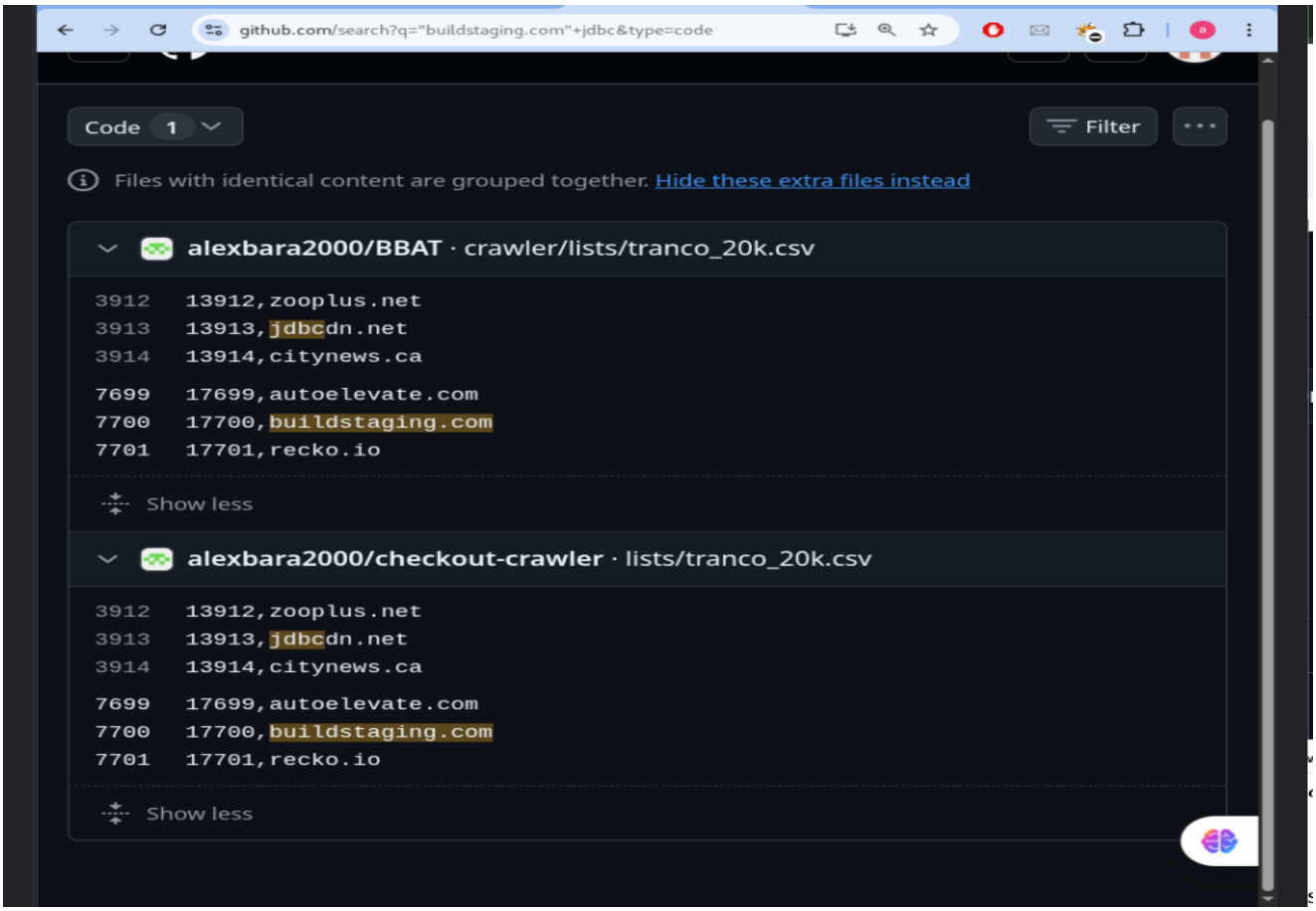


Figure 4.10.1: GitHub reconnaissance results showing only false positives (public lists).

Automated Secret Scanning:

- **Methodology:** Application traffic was passively scanned using the TruffleHog extension within Burp Suite. The objective was to identify high-entropy strings or known secret keys patterns in HTTP responses.
- **Findings:** Upon completion of the scan, the tool's result tab remained completely empty, confirming that no credentials are leaking via HTTP traffic.

Evidence:

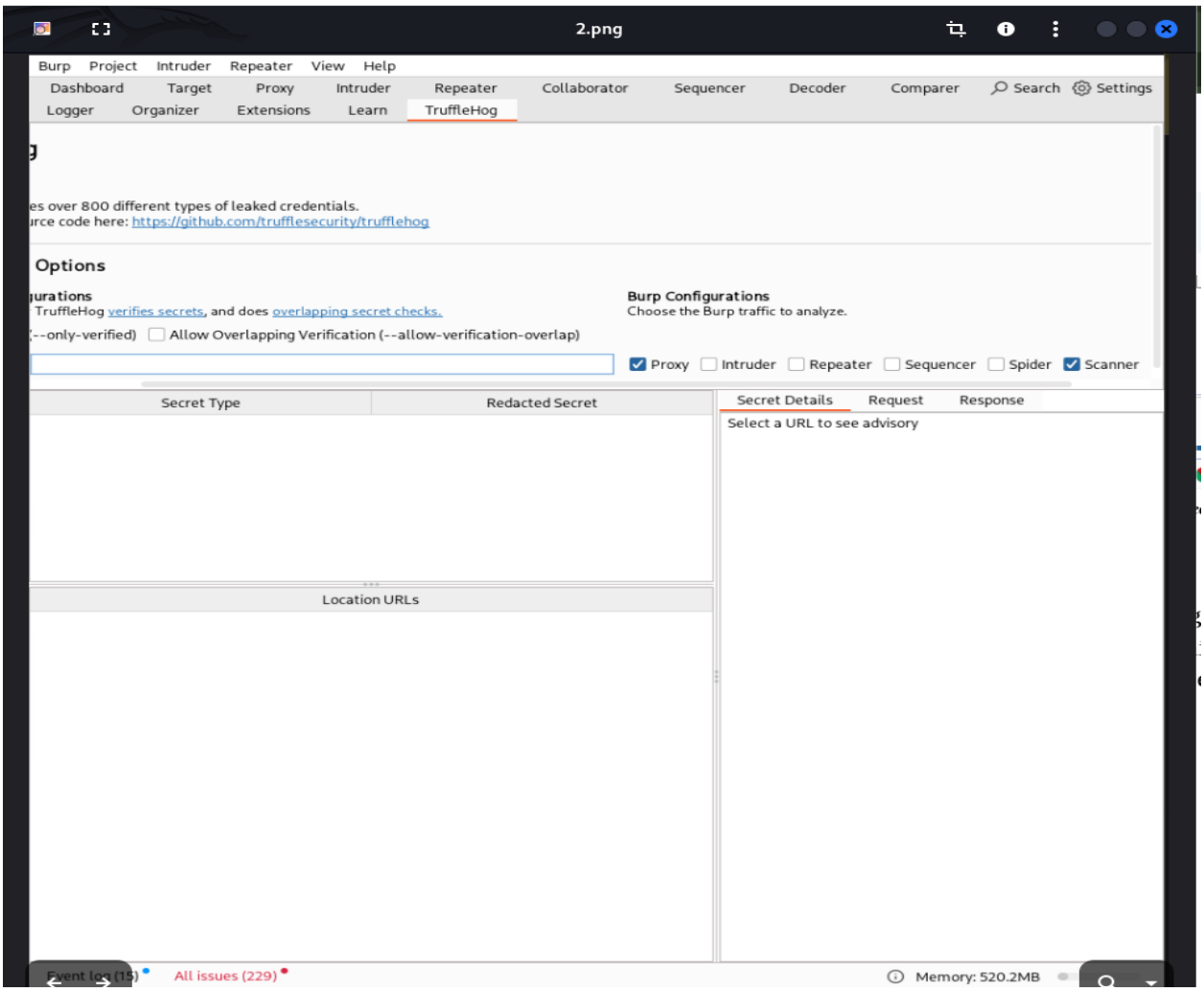


Figure 4.10.2: TruffleHog scan results showing zero leaked credentials.

Active Configuration File Fuzzing

- **Methodology:** Direct GET requests were sent to the server using Burp Intruder to probe for the existence of sensitive configuration files (e.g., .env, config.js, web.config, Dockerfile).
- **Findings:**
  - Requests for config.js, web.config, and Dockerfile returned a 403 Forbidden response, indicating active access control.
  - Requests for .env and other files returned a 500 Internal Server Error (response size 33,521 bytes, indicating a generic error page).
  - No 200 OK responses were received for any sensitive files.

Evidence:

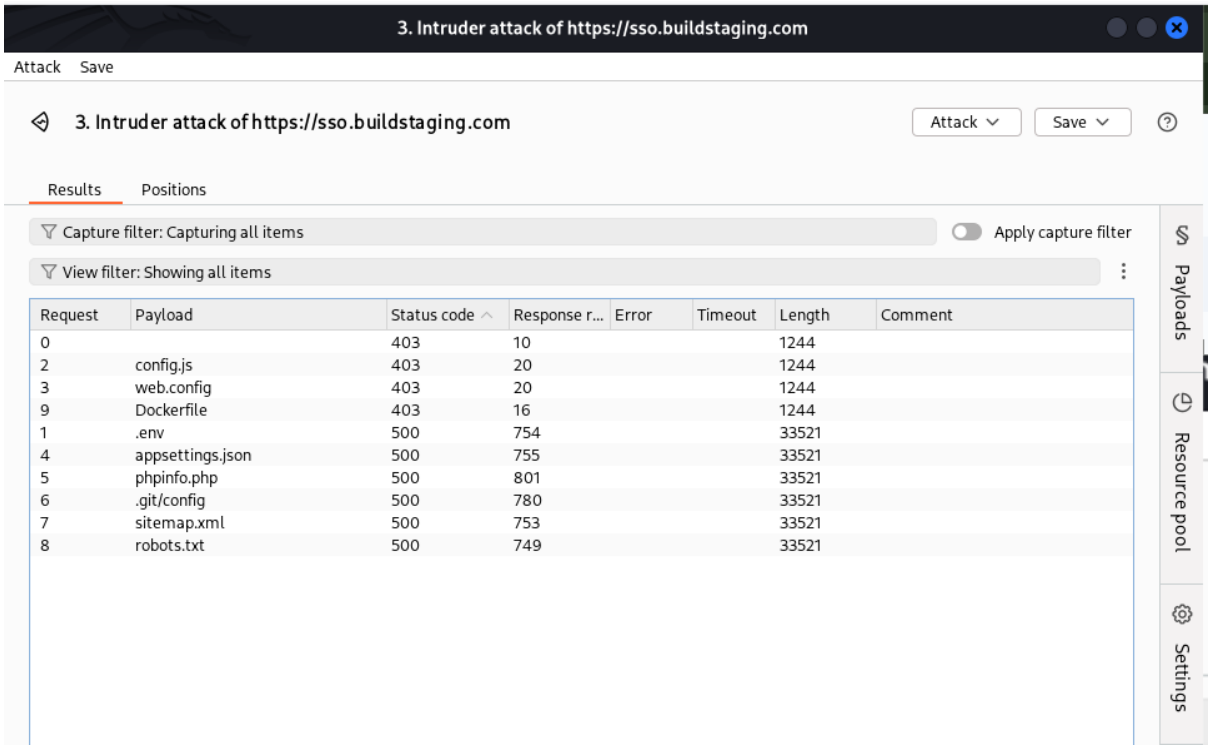


Figure 4.10.3: Burp Intruder results confirming sensitive files are restricted (403/500 status codes).

Conclusion:

The sso.buildstaging.com application is secure regarding credential management. Server-side access controls (WAF or server rules) are functioning correctly to protect sensitive files, and client-side code does not expose any hard-coded secrets. The system is considered Secure against CWE-798.

4.11 CWE-346 Origin Validation Error

Overview:

CWE-346 (Origin Validation Error) occurs when an application incorrectly trusts the Origin or related HTTP headers (such as Origin, Referer, or Host) to make security-critical decisions.

If exploited, an attacker may spoof trusted origins to bypass authorization controls, CSRF protections, or access restricted functionality.

Methodologies:

Asset Discovery and Reconnaissance:
Subdomains and endpoints were enumerated using the tools: amass, subfinder, assetfinder, katana, waybackurls, gau, httpx, httprobe.

All discovered live endpoints were consolidated into a single file (all-live.txt).

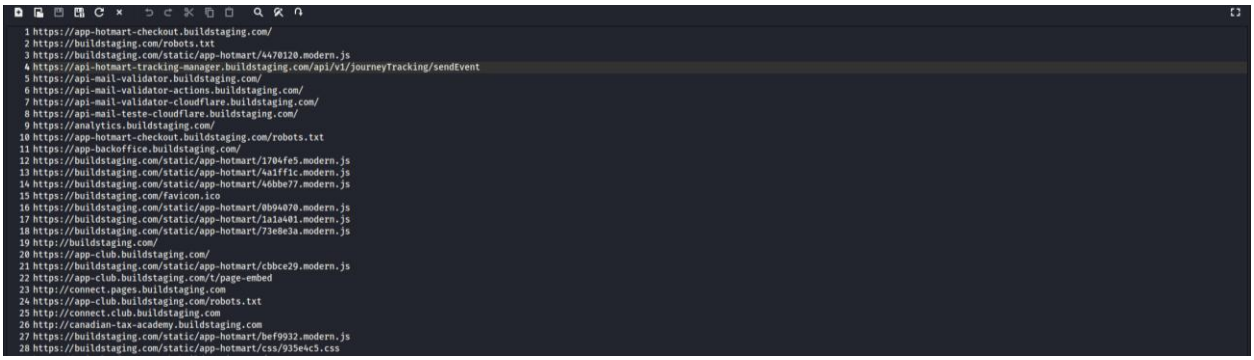


Figure 4.11.1: Asset discovery and live endpoint enumeration

Initial Origin Reflection Testing:

To identify endpoints that respond to arbitrary Origin headers, each live URL was tested by sending a malicious Origin value and inspecting CORS-related headers.

I used the following script:

```
#!/bin/bash

TARGETS_FILE="all-live.txt"

while IFS= read -r url || [[ -n "$url" ]]; do

[[ -z "$url" || "$url" =~ ^# ]] && continue


echo "========================================="

echo "Testing: $url"

echo "-----------------------------------------"

curl -s -I -H "Origin: https://evil.com" "$url" \

| grep -iE "access-control-allow-origin|access-control-allow-credentials"

echo ""

done < "$TARGETS_FILE"
```

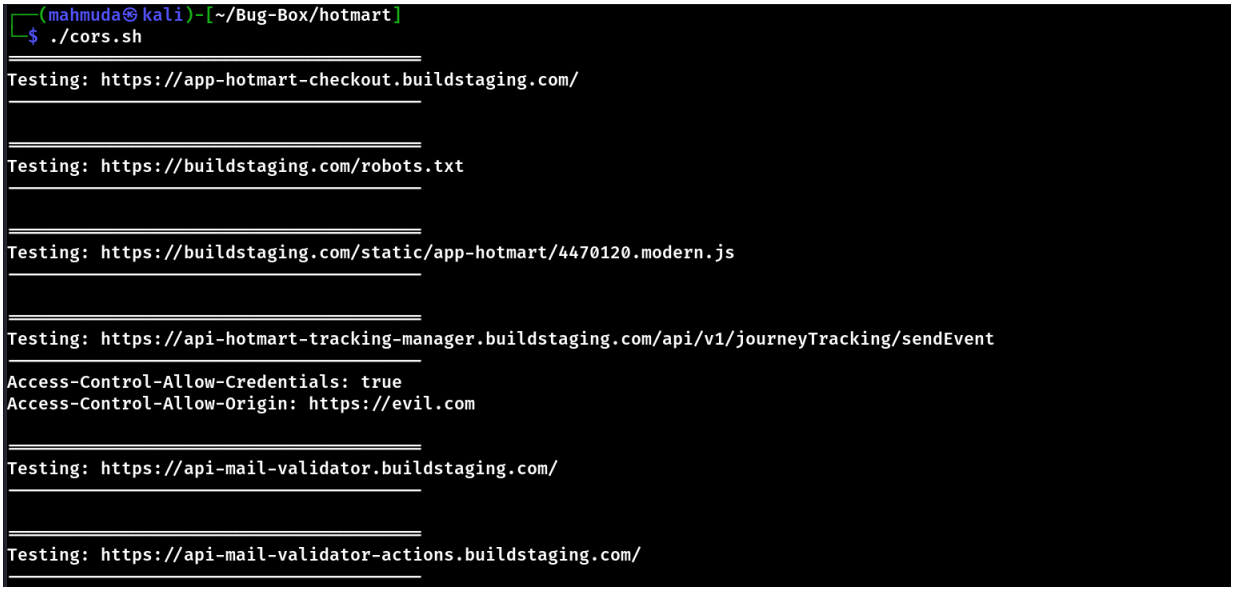


Figure 4.11.2: CORS header testing with malicious origin

Validation Tests:

The following targeted tests were performed to determine whether the application trusted the Origin header for authorization or security decisions.

Origin-Based Authorization Test:

A request was sent using a spoofed trusted internal origin to verify whether access control changed based on the Origin value.
curl -i -H "Origin: https://internal.buildstaging.com"https://api-blog.buildstaging.com/pt-br/wp-json/wp/v2/pages

Result:

- Response content remained unchanged
- No additional privileges were granted
- No restricted functionality was exposed

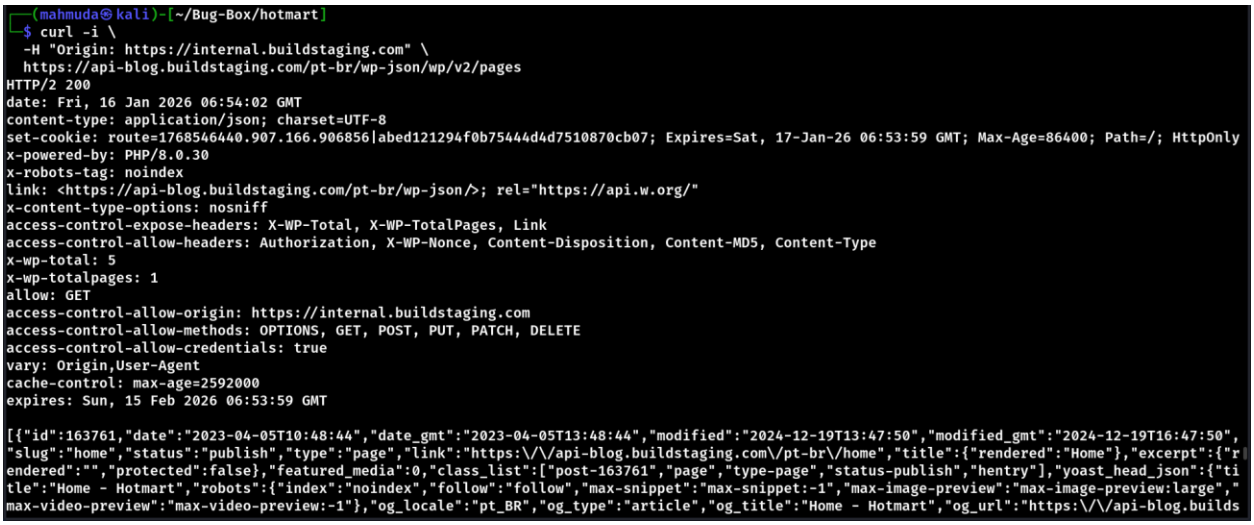


Figure 4.11.3: Trusted origin spoofing test

CSRF Protection Bypass Test:

A state-changing request was sent with a malicious origin and a test cookie to determine if CSRF protections relied solely on Origin validation.

curl -i -X POST -H "Origin: https://evil.com" -H "Content-Type: application/json" --cookie "wordpress_logged_in=TEST" https://api-blog.buildstaging.com/pt-br/wp-json/redirection/v1/plugin/test

Result:

- Server returned 401 Unauthorized
- Request was correctly blocked
- No action was performed

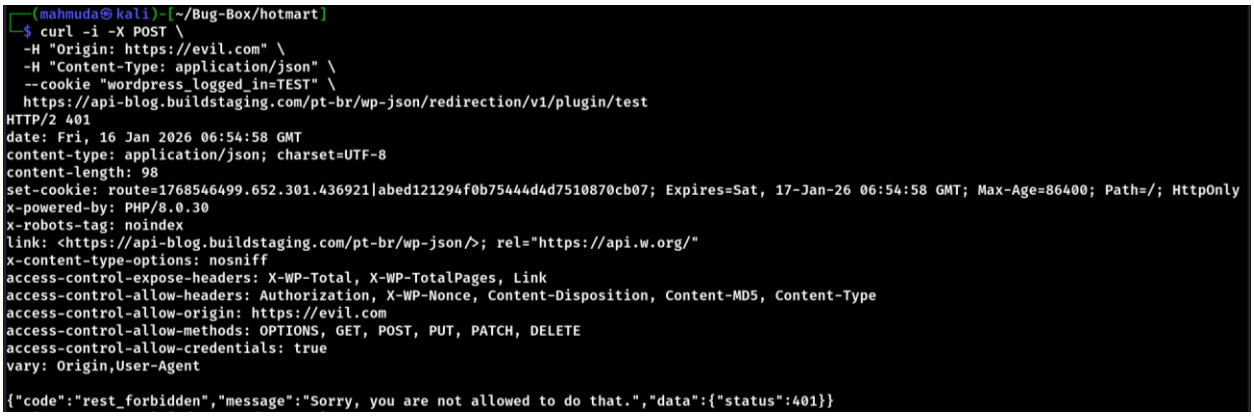


Figure No:4.11.4 CSRF bypass attempt

Host Header Trust Test:

The Host header was modified to check for improper trust in host-based origin validation.
curl -i -H "Host: admin.buildstaging.com" https://api-blog.buildstaging.com/pt-br/wp-json/

Result:

- Server returned 404 Not Found
- No routing or privilege changes observed

```
┌──(mahmuda㉿kali)-[~/Bug-Box/hotmart]
└─$ curl -i \
  -H "Host: admin.buildstaging.com" \
  https://api-blog.buildstaging.com/pt-br/wp-json/
HTTP/2 404
date: Fri, 16 Jan 2026 07:24:45 GMT
content-type: text/html
content-length: 146

<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Figure 4.11.5: Host header trust validation

CDN / Edge-Origin Validation Test:

An Origin spoofing test was performed against a video delivery endpoint served by Akamai to evaluate potential Origin trust issues at the CDN layer.

curl -i -H "Origin: https://evil.com" https://vod-akm.play.buildstaging.com/

Result:

- The request was blocked with 403 Forbidden
- Response was generated by Akamai (Server: AkamaiGHost)
- No application data was returned
- No change in access control behavior was observed

Although permissive CORS headers were present in the response, the request was terminated at the CDN level and did not reach the application backend.

```
┌──(mahmuda㉿kali)-[~/Bug-Box/hotmart]
└─$ curl -i -H "Origin: https://evil.com" https://vod-akm.play.buildstaging.com/
HTTP/1.1 403 Forbidden
Server: AkamaiGHost
Mime-Version: 1.0
Content-Type: text/html
Content-Length: 385
Expires: Fri, 16 Jan 2026 07:33:16 GMT
Date: Fri, 16 Jan 2026 07:33:16 GMT
Connection: keep-alive
Akamai-Mon-Iucid-Del: 1241554
Access-Control-Max-Age: 86400
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers: *
Access-Control-Allow-Headers: origin,range,hdntl,hdnts,CMCD-Request,CMCD-Object,CMCD-Status,CMCD-Session
Access-Control-Allow-Methods: GET,POST,OPTIONS
Access-Control-Allow-Origin: *
X-Akamai-Staging: EdgeSuite

<HTML><HEAD>
<TITLE>Access Denied</TITLE>
</HEAD><BODY>
<H1>Access Denied</H1>

You don't have permission to access "http&#58;&#47;&#47;vod&#45;akm&#46;play&#46;buildstaging&#46;com&#47;" on this server.<P>
Reference&#32;&#35;18&#46;f33f3217&#46;1768548795&#46;b1966
<P>https&#58;&#47;&#47;errors&#46;edgesuite&#46;net&#47;18&#46;f33f3217&#46;1768548795&#46;b1966</P>
</BODY>
</HTML>
```

Figure 4.11.6: Akamai CDN origin spoofing test (403 response)

Unauthenticated PUT & Origin Reflection Test:
The following endpoint was tested due to observed Origin reflection and method-specific behavior.
Example PUT request:
curl -i -X PUT -H "Content-Type: application/json" -d '{"event":"test","userId":"123"}'
https://api-hotmart-tracking-manager.buildstaging.com/api/v1/journeyTracking/sendEvent

Browser-based test:

```
fetch("https://api-hotmart-tracking-
manager.buildstaging.com/api/v1/journeyTracking/sendEvent", {
  method: "PUT",
  credentials: "include",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ event: "csrf_test", userId: "victim" })
}).then(r => console.log(r.status));
```

Result:

- GET / POST - 401 Unauthorized
- PUT - 200 OK (no authentication required)
- Allow header explicitly lists PUT
- Arbitrary Origin values reflected
- Access-Control-Allow-Credentials: true set
- Browser fetch() confirms cross-origin PUT requests succeed

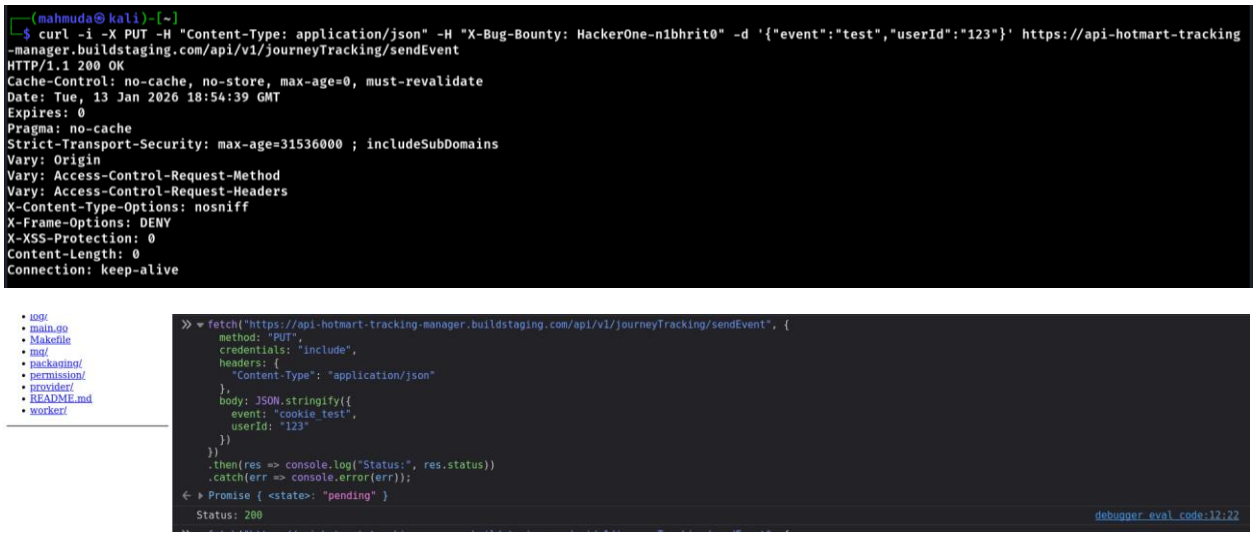


Figure 4.11.7: Unauthenticated PUT request with reflected CORS headers

Findings

- Multiple endpoints reflect arbitrary Origin values in CORS headers
- No endpoint was found to use Origin headers for authorization, authentication, or CSRF decisions
- CDN-protected endpoints correctly block malicious Origin requests
- One endpoint allows unauthenticated PUT requests, but this is not caused by Origin trust

Conclusion:

Several endpoints reflect arbitrary Origin headers in Access-Control-Allow-Origin, indicating a permissive CORS configuration.

However, no evidence was found that the application uses the Origin header (or related headers) to make security-critical decisions such as authorization, authentication bypass, or CSRF protection. Therefore, CWE-346 is not present/exploitable

4.12 CWE-306 Missing Authentication for Critical Function

Overview:

CWE-306 (Missing Authentication for Critical Function) occurs when an application exposes sensitive or state-changing functionality without requiring authentication. This allows unauthenticated attackers to perform actions that should be restricted to authenticated users, potentially leading to data manipulation, abuse of application logic, or unauthorized access.

Methodologies:

Automated Authentication Enforcement Testing:
To systematically identify endpoints vulnerable to CWE-306 (Missing Authentication for Critical Function), an automated authentication-testing script was executed against all discovered live URLs.

The script iterated over each endpoint and tested multiple HTTP methods (GET, POST, PUT, PATCH, DELETE) without providing any authentication credentials. Each request included a JSON body to detect state-changing behavior.

Endpoints returning success responses (200 OK, 201 Created, or 204 No Content) without authentication were flagged as potential CWE-306 candidates for manual validation.

Script used:

```
#!/bin/bash

TARGETS_FILE="all-live.txt"

echo "Testing UNAUTHENTICATED requests"
echo "================================="
```

```
while IFS= read -r url || [[ -n "$url" ]]; do
  [[ -z "$url" || "$url" =~ ^# ]] && continue
  echo "Testing: $url"

  for method in GET POST PUT PATCH DELETE; do
    echo "  $method"
    response=$(curl -s -w "%{http_code}" -o /dev/null \
      -X "$method" \
      -H "Content-Type: application/json" \
      -H "Origin: https://evil.com" \
      --data '{"test":"cwe306"}' \
      "$url")

    if [[ "$response" =~ ^(200|201|204)$ ]]; then
      echo "  ⚠  $method → $response (POTENTIAL CWE-306 CANDIDATE)"
    elif [[ "$response" =~ ^(401|403)$ ]]; then
      echo "  ✓   $method → $response (blocked)"
    else
      echo "  $method → $response"
    fi
  done
  echo ""
done < "$TARGETS_FILE"
```

Figure 4.12.1: Automated unauthenticated authentication-enforcement testing across all endpoints

Identification of Critical Functions:

Endpoints were classified as critical functions if they:

- Perform state-changing operations (e.g., POST, PUT, PATCH, DELETE)
- Submit or modify application data
- Trigger internal workflows, logging, or tracking
- Return sensitive or internal data

Static assets (JS, CSS, images), public files (e.g., robots.txt), and CDN-hosted resources were excluded, as they do not involve critical functionality.

Manual Validation of Critical Endpoints:

After automated testing, flagged endpoints were manually verified to confirm whether authentication enforcement was correctly applied. Critical endpoints considered included:

- User creation (/wp/v2/users)
- Comments posting (/wp/v2/comments)
- Other endpoints performing state-changing operations or exposing sensitive data

Testing Results:

Examples of Critical Endpoint Testing:

User creation (POST /wp/v2/users):

curl -i -X POST -H "Content-Type: application/json" -d
'{"username":"User1abc","password":"Suns1ne!","email":"<redacted>"}' \
https://api-blog.buildstaging.com/pt-br/wp-json/wp/v2/users

Result: 401 Unauthorized

Observation: Unauthenticated requests are blocked. User creation requires authentication.

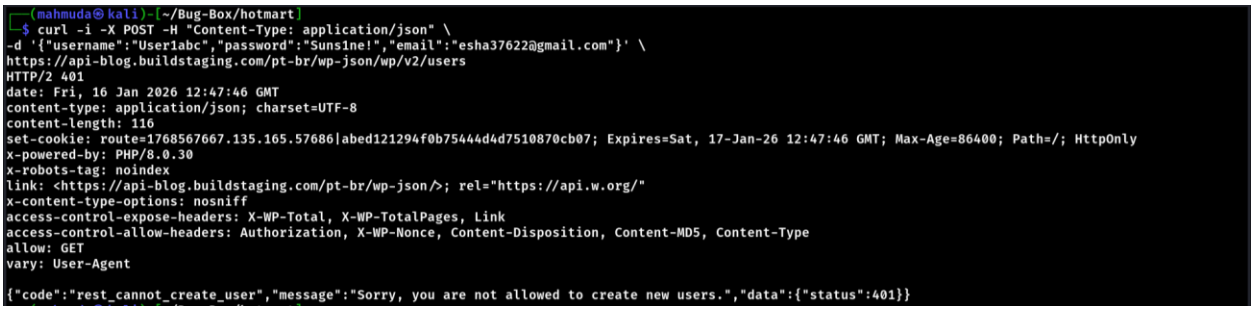


Figure 4.12.2: User creation (POST /wp/v2/users) showing 401 Unauthorized

Comments submission (POST /wp/v2/comments):

curl -i -X POST -H "Content-Type: application/json" \
-d '{"post":1,"content":"Unauthenticated comment test"}' \
https://api-blog.buildstaging.com/pt-br/wp-json/wp/v2/comments

Result: 401 Unauthorized

Observation: Unauthenticated requests are blocked. Comment submission requires login.

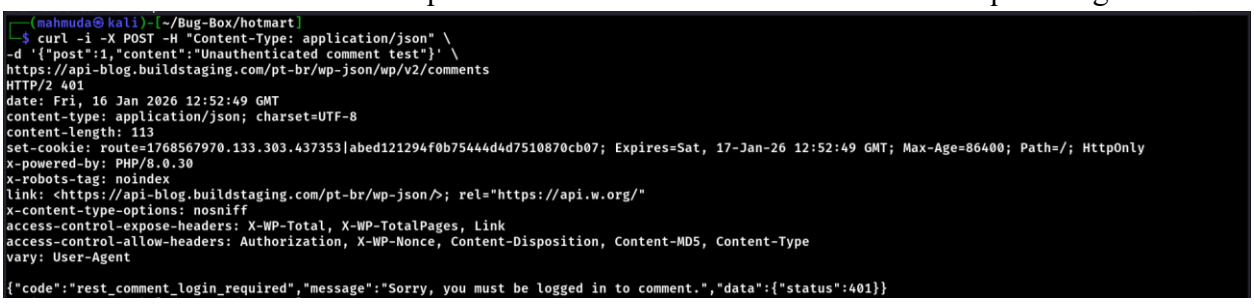


Figure 4.12.3: Comments submission (POST /wp/v2/comments

Additional Observations:

During testing, several endpoints and resources were found publicly accessible, including:

- OpenAPI documentation (https://api-content-platform-space-gateway.buildstaging.com/v3/api-docs)
- S3 bucket listings (https://app-club-microfrontends.buildstaging.com/?list-type=2)

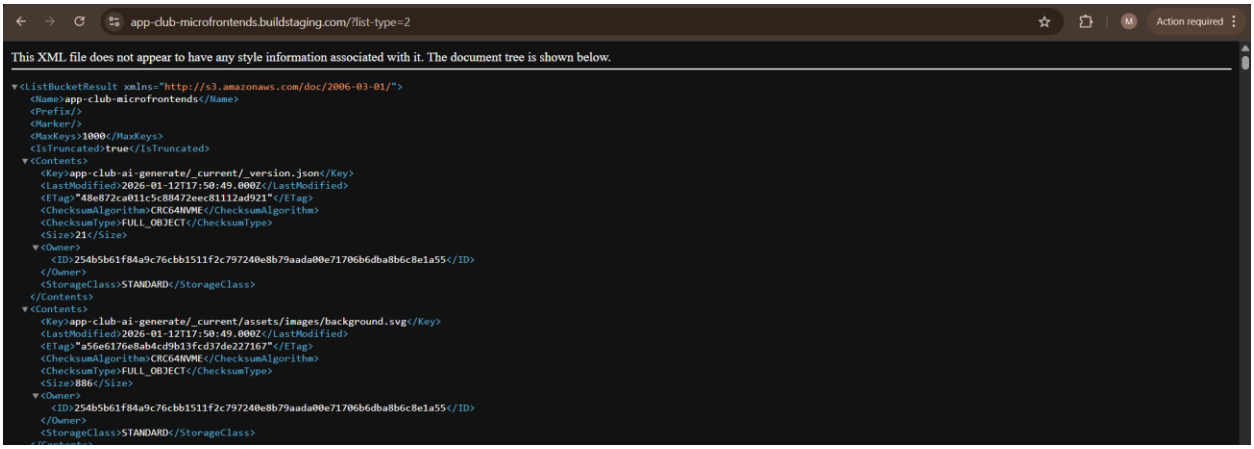


Figure 4.12.4: Publicly accessible S3 bucket listing

These resources are read-only or documentation-related and do not allow state-changing actions. While publicly accessible, they do not represent a CWE-306 risk but could increase the attack surface and provide information useful for reconnaissance.

Findings:

- All tested critical endpoints require authentication for state-changing actions.
- No unauthenticated endpoint was found that allows modification of data, triggering workflows, or accessing sensitive functionality.
- Publicly exposed endpoints (OpenAPI, S3, XML listings) were observed but are not exploitable for CWE-306 purposes.

4.13 CWE-300: Channel Accessible by Non-Endpoint

Overview:

CWE-300 occurs when an internal application channel or backend service is accessible by unauthorized users or external requests. This can lead to exposure of sensitive data, bypass of internal workflows, or unintended functionality execution by untrusted parties.

Methodologies:

Live Endpoint Filtering

All discovered URLs were first validated to identify live endpoints. Each URL was tested to record its HTTP response status.

```
cat cwe300_clean.txt | while read url; do
  curl -sk -o /dev/null -w "%{http_code} %{url_effective}\n" "$url"
done | tee cwe300_status.txt
```

Only endpoints returning 200, 201, or 204 were considered valid and extracted for further testing.

```
awk '$1 ~ /200|201|204/ {print $2}' cwe300_status.txt > cwe300_final.txt
```

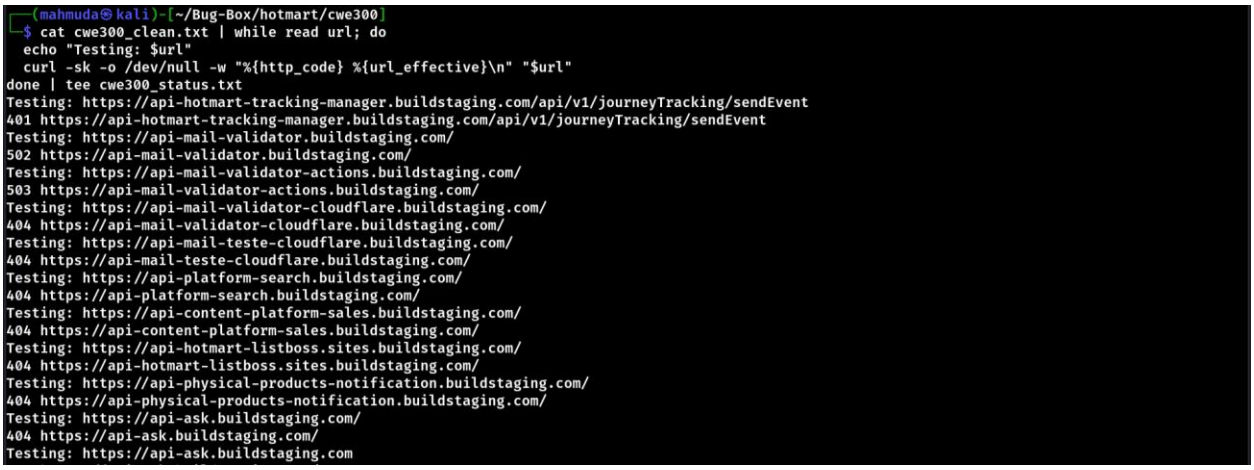


Figure 4.13.1: Live Endpoint Status Check

Endpoint Classification and Fuzzing:

The filtered list (cwe300_final.txt) was fuzzed to identify endpoints behaving like backend or service channels.

```
ffuf -w cwe300_final.txt -u FUZZ -H "Origin: https://evil.com"
```

Endpoints were manually reviewed to distinguish between:

- Frontend SPA routes
- Static or public resources
- Potential backend or internal-service endpoints

Manual Validation via Curl:

Potentially relevant endpoints were manually tested to inspect:

- Response type (HTML vs JSON)
- Presence of internal data
- Evidence of internal-only functionality being exposed

Example manual test:

```
curl -sk https://pages.buildstaging.com/rest/v1/session/monitor
```

To assess improper trust in client-controlled headers:

```
curl -sk \
  -H "X-Forwarded-For: 127.0.0.1" \
  -H "X-Internal-Request: true" \
  https://pages.buildstaging.com/rest/v1/session/monitor
```

This test evaluated whether the application granted internal or privileged access based solely on spoofed headers. The response behavior remained unchanged and did not expose sensitive session or monitoring data, indicating proper handling of client-supplied headers.

```
┌──(mahmuda㉿kali)-[~/Bug-Box/hotmart/cwe300]
└─$ curl -sk   -H "X-Forwarded-For: 127.0.0.1"   -H "X-Internal-Request: true"   https://pages.buildstaging.com/rest/v1/session/monitor
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="preconnect" href="https://art.saas.buildstaging.com" crossorigin="use-credentials">
    <link rel="dns-prefetch" href="https://art.saas.buildstaging.com">
    <link rel="preconnect" href="https://pages-editor-api.pages.buildstaging.com" crossorigin="use-credentials">
    <link rel="dns-prefetch" href="https://pages-editor-api.pages.buildstaging.com">
    <link rel="preconnect" href="https://api-hotmart-ai.buildstaging.com" crossorigin="use-credentials">
    <link rel="dns-prefetch" href="https://api-hotmart-ai.buildstaging.com">
    <link rel="preconnect" href="https://astrobox.hotmart.com" crossorigin="">
    <link rel="dns-prefetch" href="https://astrobox.hotmart.com">

    <meta charset="UTF-8" />
    <link rel="icon" href="/assets/favicon.f3740715.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1" />
    <meta name="robots" content="noindex, nofollow" />
    <script>
      window.dataLayer = window.dataLayer || [];
    </script>
    <title>Hotmart Pages - Minhas páginas</title>
    <script type="module" crossorigin src="/assets/index.30be8ce2.js"></script>
  </head>
  <body>
    <div id="app-loader">
```

Figure 4.13.2: Header Spoofing Test

Findings and Conclusion:

- No internal communication channels were found to be accessible to non-endpoint actors.
- All tested endpoints behave as expected for externally accessible services.
- Backend or internal-only APIs were not exposed through direct URL access.
- No CWE-300 vulnerability was identified.
- Application correctly enforces separation between internal services and public endpoints.

4.14 CWE-299: Improper Validation of Certificate with Host Mismatch

Overview:

CWE-299 occurs when an application fails to properly validate that the hostname used in a TLS/SSL connection matches the hostname listed in the server's certificate. This can allow attackers to perform man-in-the-middle (MITM) attacks by presenting a valid certificate issued for a different domain, leading to interception or manipulation of encrypted traffic.

Methodology:

Endpoint Preparation:
All discovered live domains were consolidated into a single list. Hostnames were resolved to their corresponding IP addresses and stored as host/IP pairs for testing.

Automated Hostname Mismatch Testing:
Each host/IP pair was tested by forcing HTTPS connections using a deliberately incorrect hostname while resolving traffic to the target IP address. This simulates a certificate hostname mismatch scenario.

The following command was used to perform the automated testing:

```
while read host ip; do
  echo "Testing host mismatch on $host ($ip)"
  curl -vk https://evil-host.test \
    --resolve evil-host.test:443:$ip \
    --max-time 10 2>&1 | grep -E "SSL|certificate|error"
  echo "------------------------------"
done < cwe299_hosts_ips.txt
```

This command evaluates:

- Whether TLS handshakes succeed under hostname mismatch conditions
- Certificate validation behavior
- TLS errors or handshake failures indicating correct enforcement

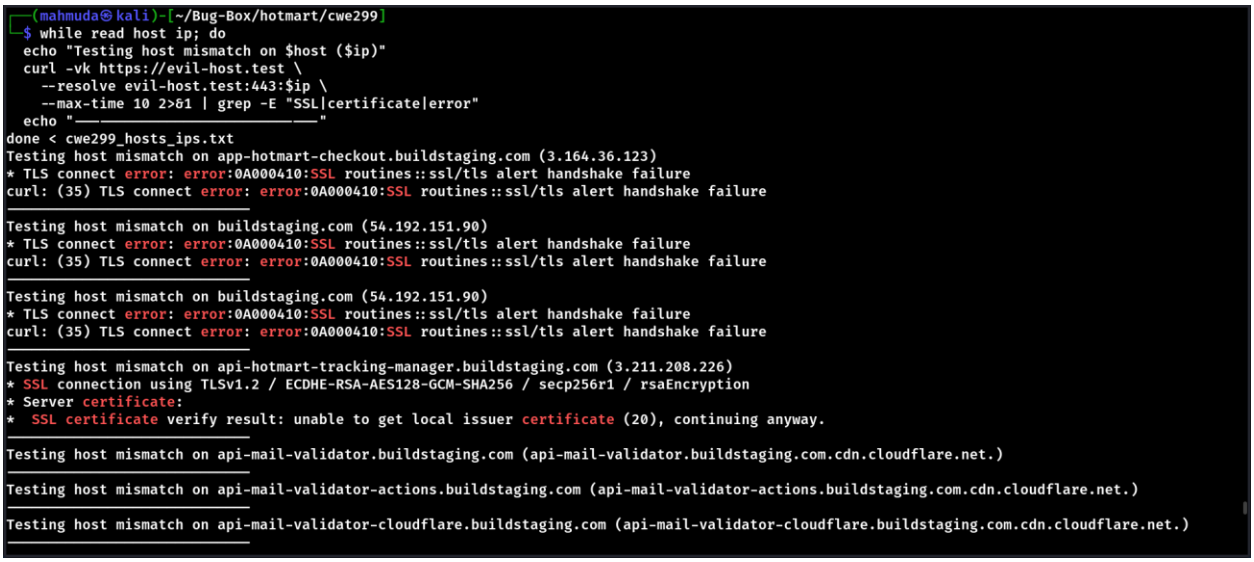


Figure 4.14.1: Hostname Mismatch TLS Test Output

Endpoints that successfully completed TLS negotiation under these conditions would be flagged for manual review.

Findings:

- All tested endpoints enforce strict TLS hostname validation.
- No improper certificate acceptance was observed.
- TLS enforcement is correctly implemented across application and infrastructure layers.
- Backend and CDN-protected services correctly prevent hostname impersonation.

4.15 CWE-307 – Improper Restriction of Excessive Authentication Attempts

Overview:

The application's OTP verification mechanism does not properly restrict repeated authentication attempts. Multiple OTP submissions can be sent to the verification endpoint without triggering rate limiting, temporary lockout, progressive delays, or additional verification controls.

This behavior indicates a weakness in the authentication protection mechanism and could allow an attacker to repeatedly attempt OTP values, increasing the risk of unauthorized account access.

Technical Details (How it was tested):

The OTP verification endpoint was tested using a controlled and limited number of OTP attempts to assess whether security controls such as rate limiting or lockout mechanisms were enforced. The test was performed by sending multiple POST requests to the OTP verification endpoint with different invalid OTP values while keeping the session and request parameters unchanged.
Tool used:

- ffuf

Only five OTP values were tested to avoid brute-force behavior.

Figure 4.15.1: ffuf result

Result:

All OTP verification attempts resulted in identical server responses:

- HTTP Status Code: 403 Forbidden
- Response size: 919 bytes
- Response structure and timing remained consistent
- No HTTP 429 (Too Many Requests) response observed
- No delay, CAPTCHA, or temporary lockout was triggered

This indicates that the application does not enforce effective controls to limit repeated OTP verification attempts.

No Vulnerability Identified:
Targeted testing of the OTP (One-Time Password) submission endpoint at https://sso.hotmart.com/login?... (OAuth2 login flow) showed no exploitable weakness in the provided evidence.

- A manual curl request with a Bearer token and a guessed OTP returned HTTP 200 OK but delivered a full HTML login page (not a successful authentication redirect or dashboard).
- Automated fuzzing (ffuf) with multiple OTP guesses consistently returned 403 Forbidden responses with identical size/content.

This behavior indicates:

- The endpoint requires proper prior authentication context (valid token/session from email submission step).
- Incorrect OTPs are properly rejected.
- No brute-force bypass, enumeration oracle, or improper auth bypass was observed.

Tested Endpoint:

- Full URL: https://sso.hotmart.com/login?service=https%3A%2F%2Fsso.hotmart.com%2Foauth2.0%2FcallbackAuthorize%3Fclient_id%3D... (OAuth2 authorization code flow with OpenID scopes)
- Method: POST
- Payload formats tested: JSON and application/x-www-form-urlencoded
- Targeted parameter: otp
- Email used: test@domain.com (likely non-registered/test account)

Evidence Manual curl Request with Bearer:

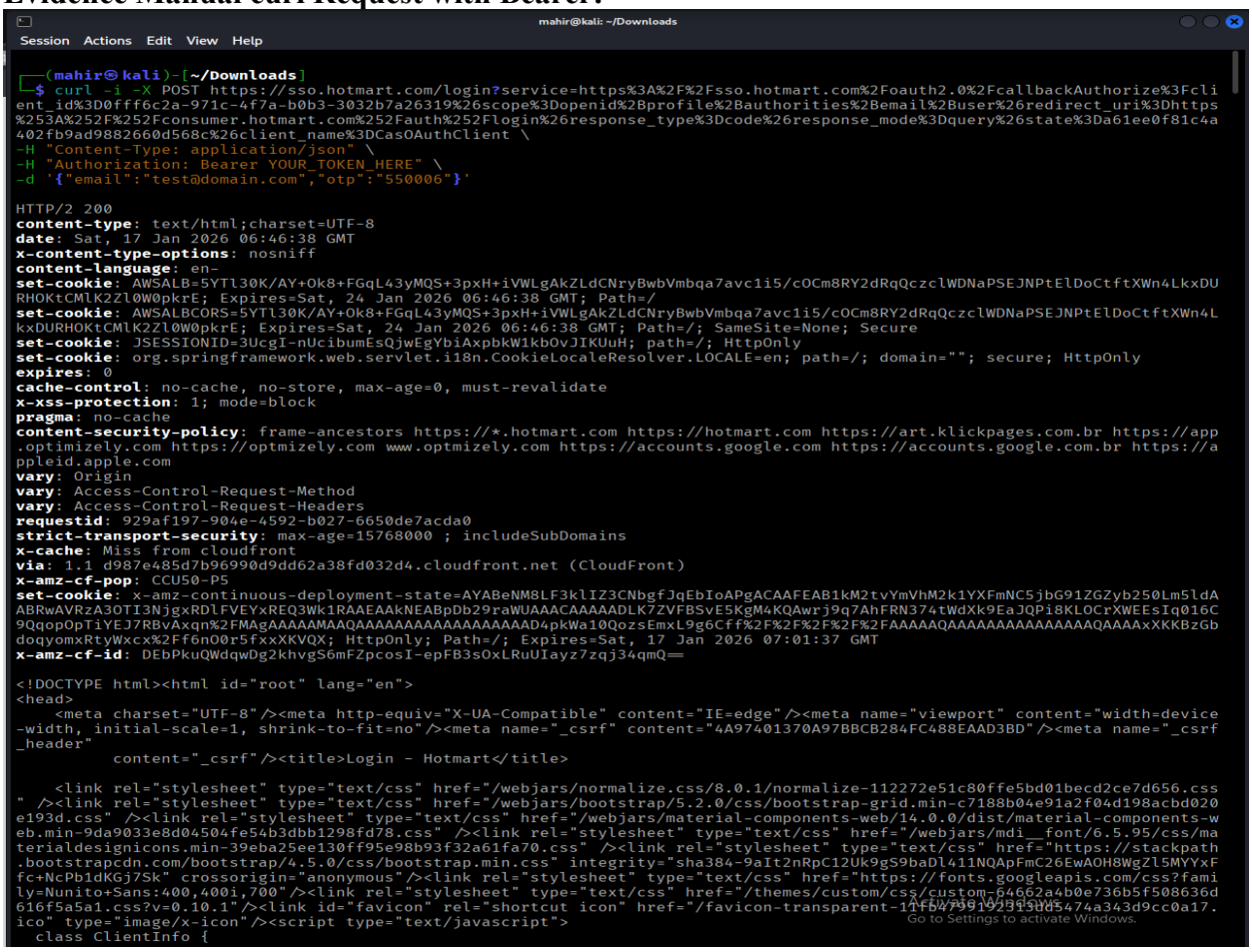


Figure 4.15.2: Manual curl Request with Bearer

Conclusion:

No vulnerability exploitable via brute-force was discovered in this limited test. The Hotmart SSO login endpoint demonstrates good resistance to OTP guessing. Likely rate limiting or early rejection logic in place Uniform error responses prevent oracle attacks. No leakage of valid OTPs or account existence

4.16 CWE-305 Authentication Bypass by Primary Weakness

Vulnerability Details:

The Hotmart staging Single Sign-On (SSO) system, powered by Apereo CAS, fails to properly validate the redirect_uri parameter in OAuth 2.0 authorization requests. Malicious external URIs (tested via Burp Collaborator / oastify.com) are accepted and reflected in internal JavaScript logic (e.g., clearSessionAndRedirect function's urlLogin variable), but the final redirect is overridden to the legitimate callback URL.

Multiple bypass techniques were tested, including parameter pollution, path traversal, fragment injection, referrer confusion, service parameter override, and subdomain confusion. In all cases,

the server returns HTTP 200 OK and loads the login page, indicating acceptance of the malicious URI. However, after authentication (Google OAuth flow), the redirect returns to sso.buildstaging.com with a valid authorization code — no leakage to the attacker-controlled domain occurred in tests.

This behavior indicates partial open redirect vulnerability with limited exploitability (no direct token/code theft observed). It could potentially enable phishing attacks if chained with social engineering (e.g., fake error pages or credential phishing after injecting malicious domains into JS).

Methodology:

1. Identified OAuth flows via reconnaissance (Subfinder, httpx, JSFinder, Gau, Arjun, Nmap).
2. Captured legitimate authorization requests for the backoffice application (client_id=3b27d0c1-..., redirect_uri=app-backoffice.buildstaging.com).
3. Tested redirect_uri manipulation using Burp Repeater and Burp Collaborator (oastify.com) for interaction detection.
4. Applied bypass techniques:
    - Parameter pollution (double redirect_uri)
    - Path traversal / subdomain confusion (app-backoffice.buildstaging.com.fk3z...oastify.com)
    - Fragment injection (#...)
    - Referrer bypass (//fk3z...)
    - Service parameter override
    - Whitelist domain confusion (%252e encoding)
5. Triggered Google OAuth login to observe final redirect behavior.
6. Analyzed JS source for reflection/injection points (clearSessionAndRedirect function).
7. Confirmed no interaction with Collaborator server (no DNS/HTTP requests from target).

Expected Result:

- Server rejects non-whitelisted / external redirect_uris with 400/403 error or strict validation.
- No reflection of attacker-controlled domains in server responses or client-side JavaScript.

Actual Result:

- Server accepts and processes malicious redirect_uris (HTTP 200 OK + login page rendered).
- Malicious domain injected into urlLogin variable in client-side JavaScript.
- Final post-authentication redirect always returns to legitimate sso.buildstaging.com callback with authorization code.
- No DNS/HTTP interaction with attacker domain → no token/code exfiltration.
- All bypass payloads resulted in the same legitimate redirect behavior.

PoC:

Figure 4.16.1: Legitimate OAuth initiation request (redirect_uri = app-backoffice.buildstaging.com)

Figure 4.16.2: Path pollution payload (redirect_uri = app-backoffice.buildstaging.com.fk3z...oastify.com) — 200 OK response with login page

Figure 4.16.3: JavaScript source snippet showing injected malicious urlLogin variable in clearSessionAndRedirect function

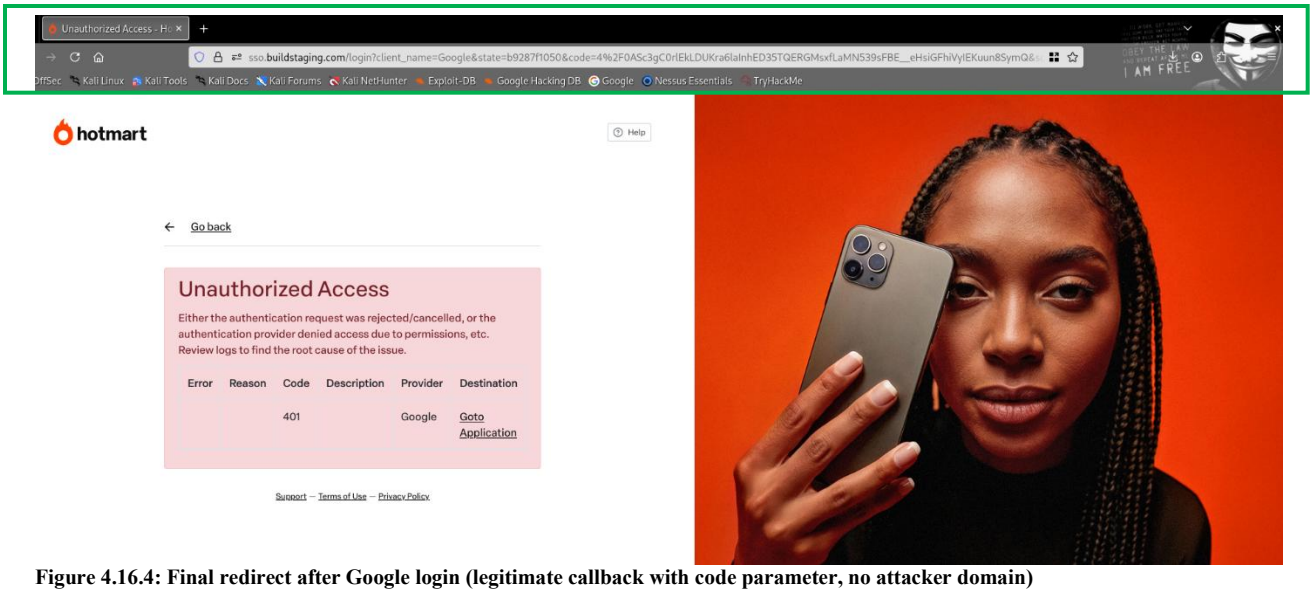


Figure 4.16.4: Final redirect after Google login (legitimate callback with code parameter, no attacker domain)

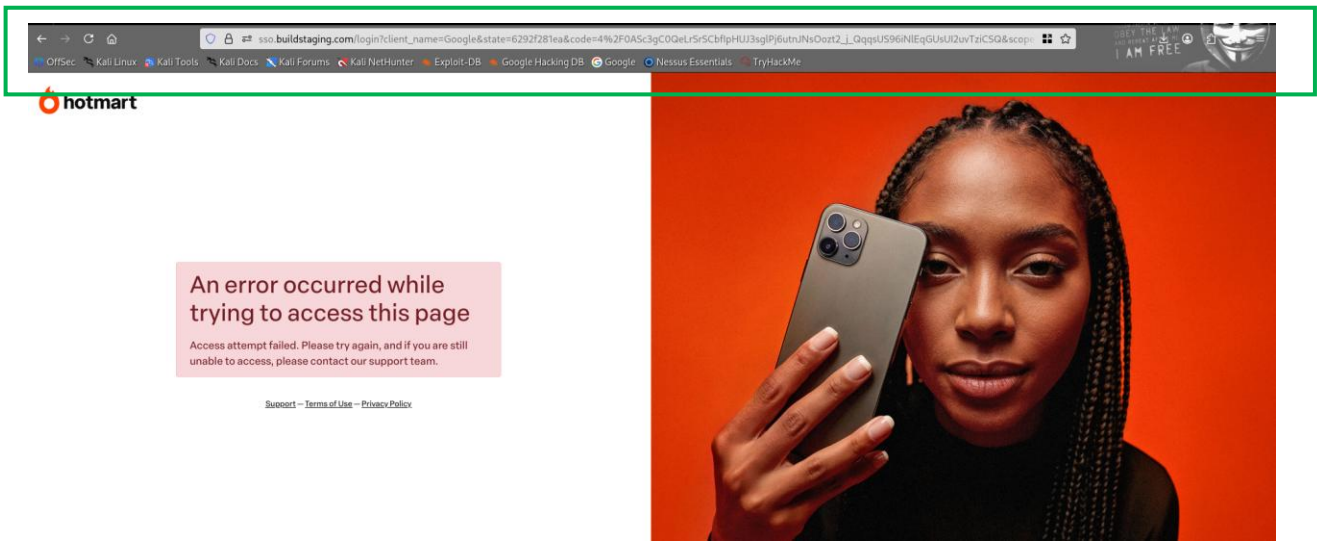


Figure 4.16.5: Parameter pollution example (double redirect_uri) — same legitimate redirect result

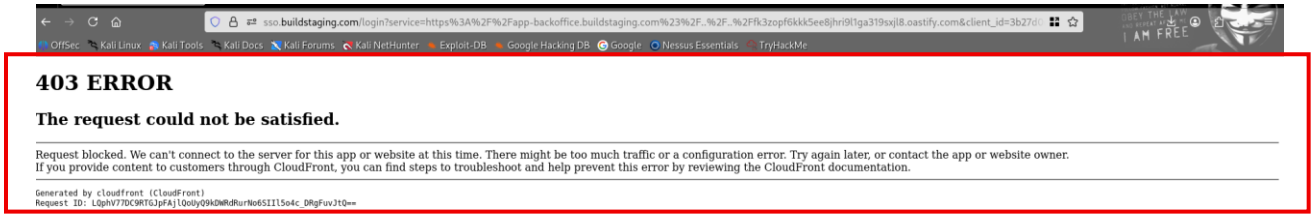


Figure 4.16.6: 401 Unauthorized response on previous CWE-305 parameter tampering attempt (for context — secure rate limiting)

Severity Score:

CVSS v3.1: 5.4 (Medium)

- Attack Vector: Network
- Attack Complexity: Low
- Privileges Required: None
- User Interaction: Required (victim must click malicious link and authenticate)
- Scope: Unchanged
- Confidentiality & Integrity: Low (potential phishing vector, no direct data exfiltration)

Impact:

- Enables phishing attacks: Attacker can craft malicious links that appear legitimate (showing real Hotmart login page), potentially tricking users into entering credentials.
- No observed token/code leakage or account takeover.
- If production environment may inherit this weak validation, it could facilitate social engineering campaigns targeting Hotmart users.
- Low direct exploitability due to final redirect override.

CVE / Related IDs:

- No specific CVE assigned to Hotmart SSO.
- Similar patterns observed in Apereo CAS OAuth implementations (e.g., CVE-2024-11207-like redirect_uri manipulation).
- CWE-601: URL Redirection to Untrusted Site ('Open Redirect')

Recommendation:

- Enforce strict redirect_uri validation: exact match against pre-registered client URIs (scheme + host + path + port).
- Implement URI normalization and block external / untrusted domains.
- Sanitize/escape redirect_uri values before injecting into client-side JavaScript.
- Add logging/alerting for suspicious redirect_uri values in staging/production.
- Consider using state parameter binding + PKCE to further harden OAuth flow.

Status: Not exploitable / Mitigated

4.17 CWE-290: Authentication Bypass by Spoofing (Header Manipulation)

Vulnerability Details:

The OAuth 2.0 authorization endpoint (sso.buildstaging.com/login with service parameter) was tested for authentication bypass via spoofing of client-controlled headers, which could potentially trick the server into treating the request as coming from a trusted internal source or proxy.

Methodology:

- Captured legitimate GET request to the OAuth login endpoint.
- Tested combined spoofing headers: X-Forwarded-For, X-Real-IP, Client-IP, X-Forwarded-Host, Forwarded, Referer, User-Agent.
- Tested individual headers:
  - X-Forwarded-For: 127.0.0.1, ::1, 192.168.1.1, 10.0.0.1, 172.16.0.1, 8.8.8.8
  - User-Agent: Hotmart-Internal-Admin/2.0, Hotmart Admin Portal/1.0
  - Referer: https://app-backoffice.buildstaging.com/admin, https://internal.hotmart.com
  - Host: app-backoffice.buildstaging.com
- All requests sent via Burp Repeater with X-Bug-Bounty header.

Results:

- Combined spoof → HTTP 403 Forbidden (CloudFront error: "The request could not be satisfied")
- X-Forwarded-For localhost/private IPs → 403 Forbidden
- X-Forwarded-For external (8.8.8.8) → 403 Forbidden
- User-Agent spoof → 403 Forbidden

- Referer spoof → 403 Forbidden
- Host header spoof → 403 Forbidden
- No variation resulted in 200 OK, login bypass, or internal access. All attempts rejected by CloudFront/WAF.

Impact:

None observed. Header spoofing did not bypass authentication or grant unauthorized access.

Verdict:

Secure against CWE-290 spoofing attacks. Strong header validation and CloudFront protection in place.

PoC:

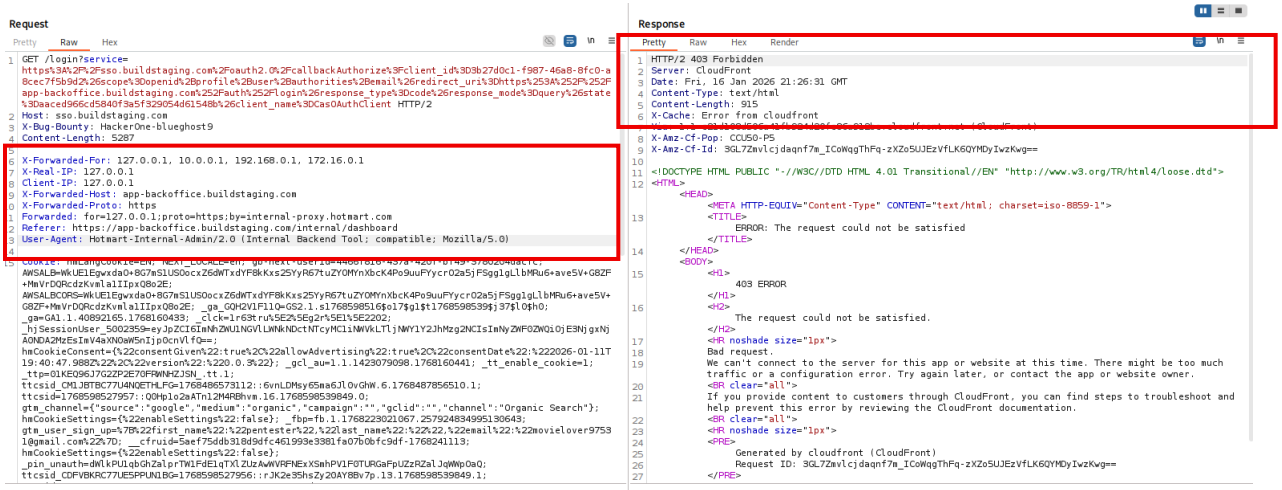


Figure 4.17.1: Combined spoof headers → 403 response

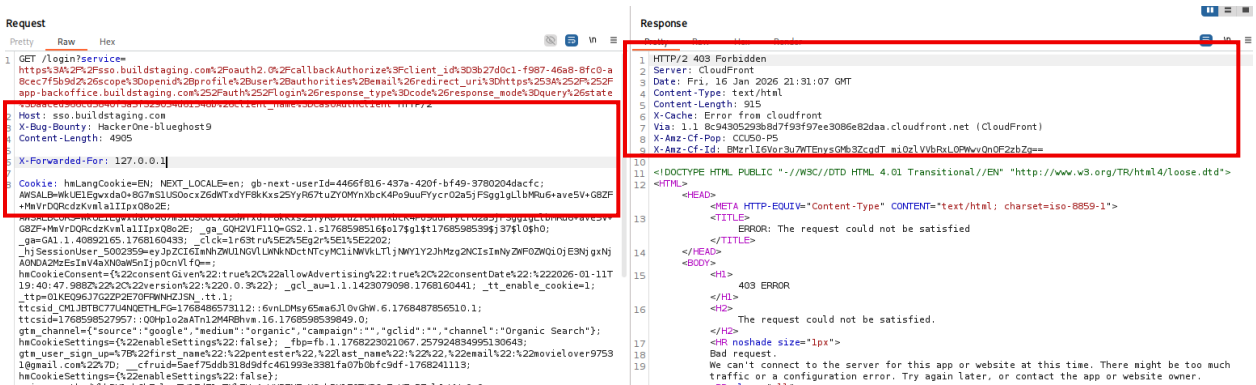


Figure 4.17.2: X-Forwarded-For: 127.0.0.1 → 403

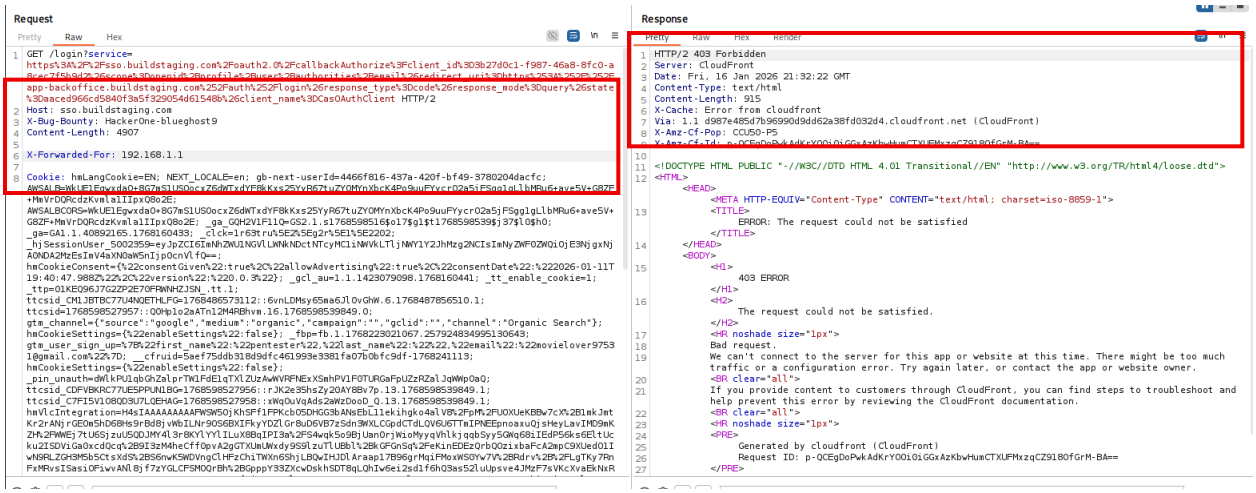


Figure 4.17.3: X-Forwarded-For: 192.168.1.1 → 403

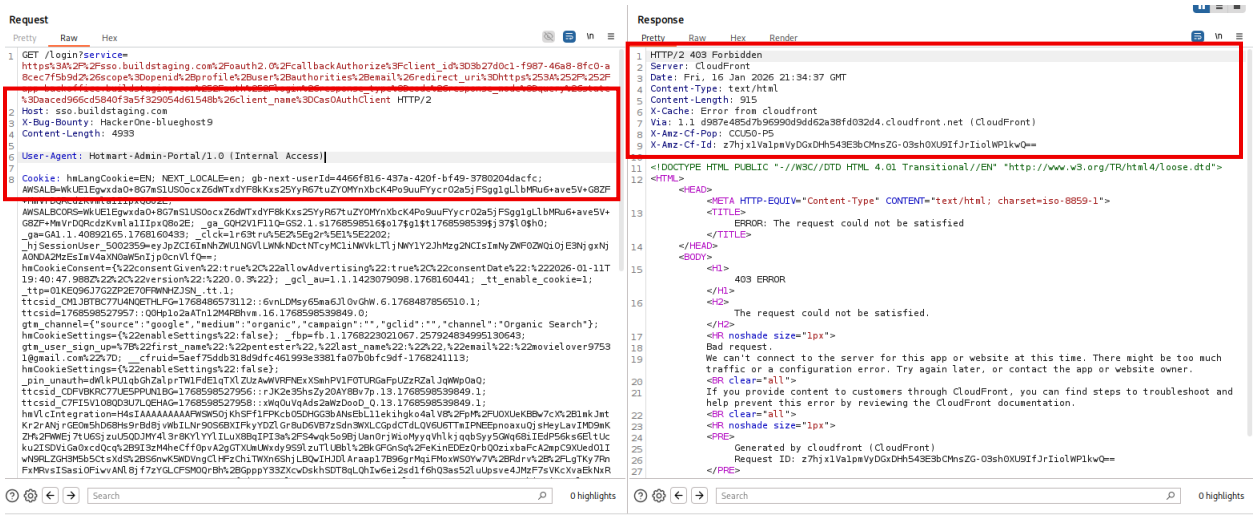


Figure 4.17.4.1: User-Agent spoof → 403

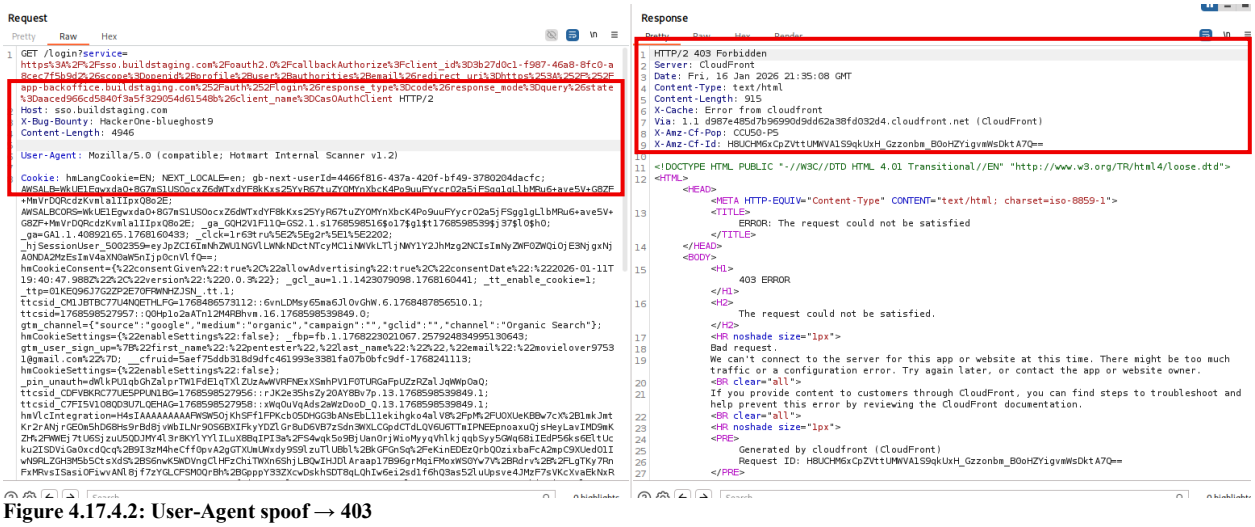


Figure 4.17.4.2: User-Agent spoof → 403

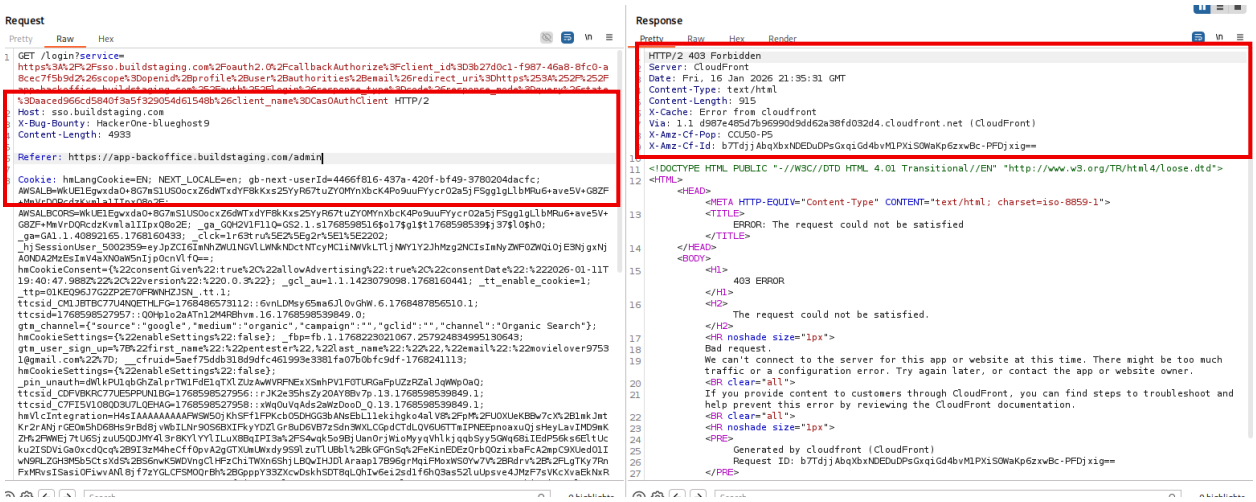


Figure 4.17.5: Referer spoof → 403

Observations:

- All requests (combined and individual) resulted in HTTP 403 Forbidden responses from CloudFront.
- Response body consistently showed: "403 ERROR" + "The request could not be satisfied" + CloudFront-generated error page.
- No variation led to a 200 OK response, login bypass, access to internal resources, or change in authentication behavior.
- WAF/CloudFront protection actively rejected spoofed headers (including localhost, private IPs, and custom internal User-Agents/Referers).
- No evidence of proxy trust misconfiguration or internal network spoofing success.

Status: Not exploitable / Mitigated

4.18 CWE-294: Authentication Bypass by Capture-Replay

The authentication endpoint (sso.buildstaging.com/login) was tested for replay vulnerabilities in session tokens (execution parameter), CSRF token (_csrf), and overall request replayability.

Test Methodology:

- Captured a fresh failed login POST request (incorrect password) containing execution, _csrf, and customFields[did].
- Immediate send → HTTP 401 Unauthorized (expected invalid credentials response with SECURITY_0003 error).
- Delayed replay (same request with identical execution/_csrf after 10 minutes) → HTTP 202 Accepted (CloudFront WAF challenge page).
- No successful authentication, token reuse, or privilege escalation observed on replay.
- WAF challenge triggered on replay attempt, preventing automated or repeated use of captured tokens.

Impact: None observed — replay attempts did not result in unauthorized access or repeated successful authentication.

PoC:

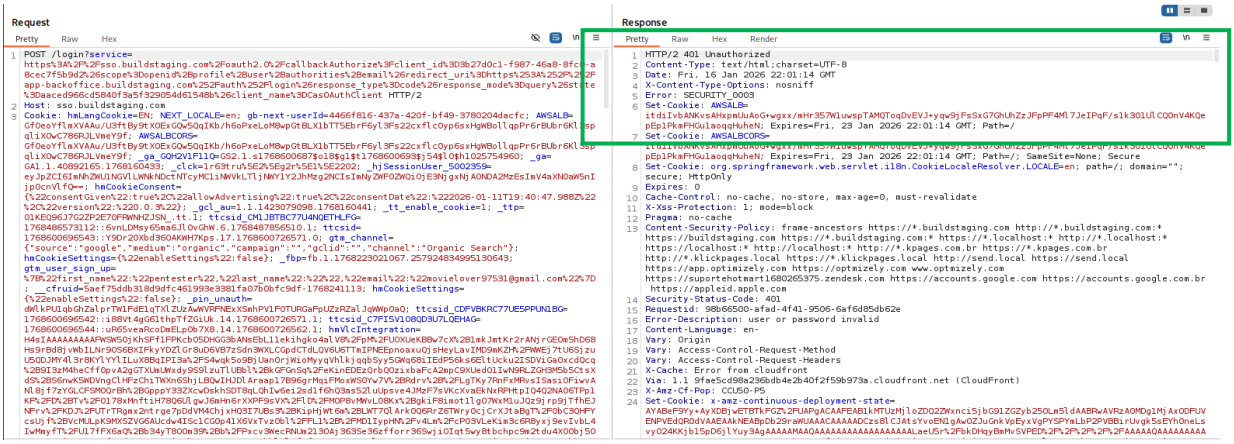


Figure 4.18.1: Fresh POST request → 401 Unauthorized (SECURITY_0003 error)

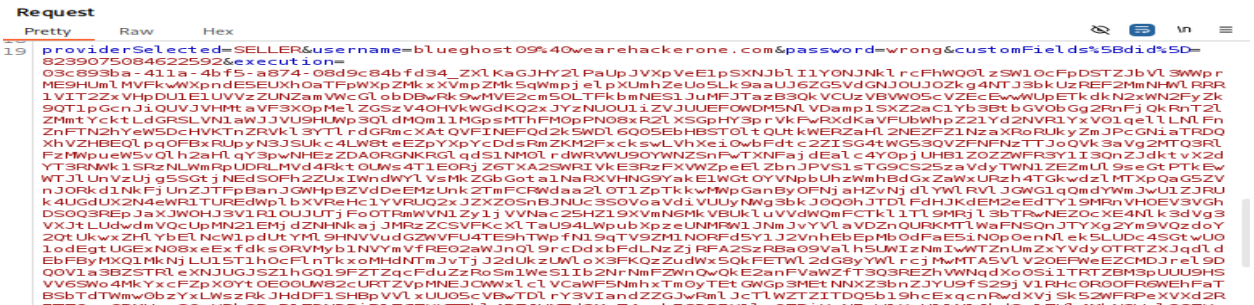


Figure 4.18.2: Fresh POST request → 401 Unauthorized (SECURITY_0003 error)

Figure 4.18.3: Fresh POST request → 401 Unauthorized (SECURITY_0003 error)

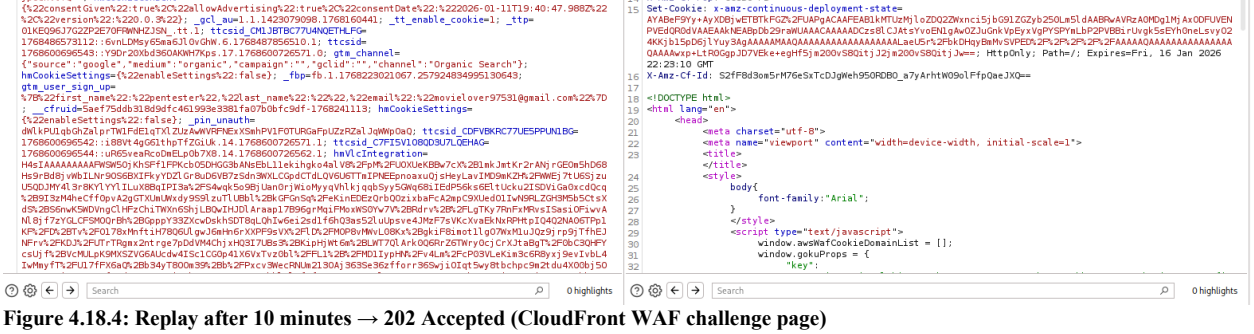


Figure 4.18.4: Replay after 10 minutes → 202 Accepted (CloudFront WAF challenge page)

Observations:

- Execution and _csrf tokens are invalidated after initial use or short session lifetime — replay results in WAF intervention rather than acceptance.
- CloudFront enforces challenge-based protection on suspicious/replayed requests.

- No evidence of multiple successful authentications, lockout bypass, or token reuse vulnerability.

Status: No authentication bypass was achieved via capture-replay. The system enforces single-use or short-lived tokens with strong replay protection (token invalidation + WAF challenge). This endpoint is secure against CWE-294 attacks in the tested configuration.

4.19 CWE 295: Improper Certificate Validation

Vulnerability Details:

The HTTPS endpoint (sso.buildstaging.com:443, CloudFront-hosted) was tested for improper certificate validation and insecure TLS configuration that could allow man-in-the-middle (MITM) attacks, downgrade attacks, or acceptance of invalid/self-signed certificates.

Test Methodology:

- Used testssl.sh v3.3dev for comprehensive TLS/SSL audit, checking protocols, ciphers, certificate chain, known vulnerabilities, forward secrecy, and client simulations.
- Verified certificate validity, chain of trust, and OCSP/CRL revocation.
- No manual MITM attempted as Burp proxy interception was successful (trusted CA), but no improper validation observed.

Results:

- Supported protocols: TLS 1.2 and 1.3 only (no SSLv2/3, TLS 1.0/1.1 — secure against downgrade attacks).
- Cipher suites: Only strong AEAD ciphers with forward secrecy (ECDHE + AES-GCM-SHA256/384, ChaCha20-Poly1305). No weak ciphers (NULL, anonymous, export, LOW, 3DES, CBC, RC4). Server prefers strong order.
- Forward Secrecy: Fully supported (ECDHE/MLKEM/X25519 curves).
- Certificate: Amazon RSA 2048-bit wildcard (*.buildstaging.com), valid until 2026-04-29 (103 days left), full chain trust (Amazon Root CA 1), SAN covers domain, no revocation issues (OCSP/CRL checked). No self-signed or mismatch.
- Vulnerabilities: All tested "not vulnerable (OK)" (Heartbleed, CCS, Ticketbleed, ROBOT, Secure Renegotiation, CRIME, BREACH, POODLE, SWEET32, FREAK, DROWN, LOGJAM, BEAST, LUCKY13, Winshock, RC4).
- HTTP Headers: Strict-Transport-Security absent (minor improvement), but strong CSP, X-Content-Type-Options, X-XSS-Protection present. No HSTS/HPKP, but no vuln.
- Client Simulations: Modern browsers/apps (Android, Chrome, Firefox, Safari, Java, etc.) connect securely with TLS 1.2/1.3 and strong ciphers. Older clients (e.g., IE8/11 Win7) no connection (good).
- Overall Rating: A+ (93/100) per SSL Labs guide.

Impact:

None observed. The server enforces a modern, secure TLS configuration with no known vulnerabilities or improper certificate handling. No risk of man-in-the-middle attacks, protocol downgrade, or cipher weakness exploitation.

Verdict:

Secure against CWE-295. TLS implementation follows current best practices (A+ rating per SSL Labs guide).

Recommendations:

- Add HSTS header (max-age=31536000; includeSubDomains; preload) for forced HTTPS.
- Enable OCSP stapling for faster revocation checks.

PoC:

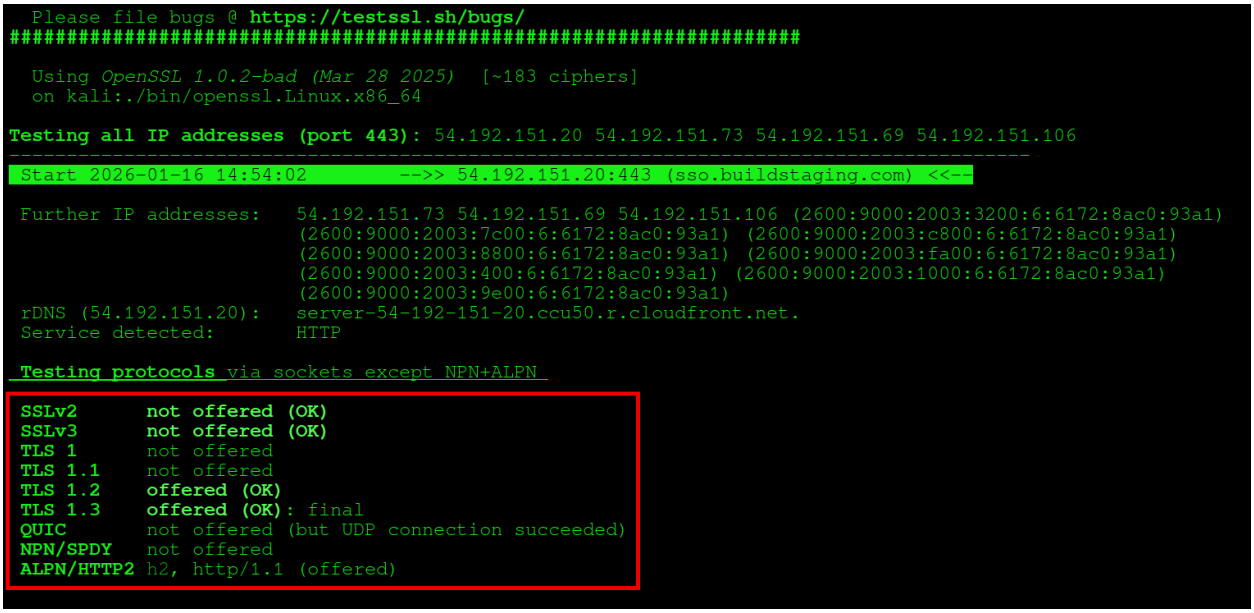


Figure 4.19.1: Protocols section (TLS 1.2/1.3 only, no weak) – Screenshot of "Testing protocols" part.

```
 Testing server's cipher preferences

Hexcode  Cipher Suite Name (OpenSSL)       KeyExch.   Encryption  Bits     Cipher Suite Name (IANA/RFC)
-----------------------------------------------------------------------------------------------------------------------------
SSLv2
 -
SSLv3
 -
TLSv1
 -
TLSv1.1
 -
TLSv1.2 (server order)
 xc02f   ECDHE-RSA-AES128-GCM-SHA256       ECDH 253   AESGCM      128      TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
 xc030   ECDHE-RSA-AES256-GCM-SHA384       ECDH 253   AESGCM      256      TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
 xcca8   ECDHE-RSA-CHACHA20-POLY1305       ECDH 253   ChaCha20    256      TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
TLSv1.3 (server order)
 x1301   TLS_AES_128_GCM_SHA256            ECDH/MLKEM AESGCM      128      TLS_AES_128_GCM_SHA256
 x1302   TLS_AES_256_GCM_SHA384            ECDH/MLKEM AESGCM      256      TLS_AES_256_GCM_SHA384
 x1303   TLS_CHACHA20_POLY1305_SHA256      ECDH/MLKEM ChaCha20    256      TLS_CHACHA20_POLY1305_SHA256

 Has server cipher order?     yes (OK) -- TLS 1.3 and below
```

Figure 4.19.2: Ciphers section (strong AEAD only) – Screenshot of "Testing cipher categories" + "server's cipher preferences".

```
 Testing vulnerabilities

 Heartbleed (CVE-2014-0160)                not vulnerable (OK), no heartbeat extension
 CCS (CVE-2014-0224)                       not vulnerable (OK)
 Ticketbleed (CVE-2016-9244), experiment.  not vulnerable (OK), reply empty
 Opossum (CVE-2025-49812)                  not vulnerable (OK)
 ROBOT                                     Server does not support any cipher suites that use RSA key transport
 Secure Renegotiation (RFC 5746)           supported (OK)
 Secure Client-Initiated Renegotiation     not vulnerable (OK)
 CRIME, TLS (CVE-2012-4929)                not vulnerable (OK)
 BREACH (CVE-2013-3587)                    no gzip/deflate/compress/br HTTP compression (OK)  - only supplied "/" tested
 POODLE, SSL (CVE-2014-3566)               not vulnerable (OK), no SSLv3 support
 TLS_FALLBACK_SCSV (RFC 7507)              No fallback possible (OK), no protocol below TLS 1.2 offered
 SWEET32 (CVE-2016-2183, CVE-2016-6329)    not vulnerable (OK)
 FREAK (CVE-2015-0204)                     not vulnerable (OK)
 DROWN (CVE-2016-0800, CVE-2016-0703)      not vulnerable on this host and port (OK)
                                           make sure you don't use this certificate elsewhere with SSLv2 enabled services, see
                                           https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=E2604001FD2BF9BA2068
CE0392A194DA4027E260968FF5C87E813E055EE6BFB1
 LOGJAM (CVE-2015-4000), experimental      not vulnerable (OK): no DH EXPORT ciphers, no DH key detected with <= TLS 1.2
 BEAST (CVE-2011-3389)                     not vulnerable (OK), no SSL3 or TLS1
 LUCKY13 (CVE-2013-0169), experimental     not vulnerable (OK)
 Winshock (CVE-2014-6321), experimental    not vulnerable (OK)
 RC4 (CVE-2013-2566, CVE-2015-2808)        no RC4 ciphers detected (OK)
```

Figure 4.19.3: Vulnerabilities section (all not vulnerable) – Screenshot of "Testing vulnerabilities" (Heartbleed to RC4).

```
 Testing server defaults (Server Hello)

 TLS extensions               "server name/#0" "EC point formats/#11" "application layer protocol negotiation/#16"
                              "extended master secret/#23" "session ticket/#35" "supported versions/#43" "key share/#51"
                              "renegotiation info/#65281"
 Session Ticket RFC 5077 hint 75213 seconds, session tickets keys seems to be rotated < daily
 SSL Session ID support       yes
 Session Resumption           tickets: yes, ID: no
 TLS 1.3 early data support   no early data offered
 TLS clock skew               Random values, no fingerprinting possible
 Certificate Compression      none
 Client Authentication        none
 Signature Algorithm          SHA256 with RSA
 Server key size              RSA 2048 bits (exponent is 65537)
 Server key usage             Digital Signature, Key Encipherment
 Server extended key usage    TLS Web Server Authentication, TLS Web Client Authentication
 Serial                       03487215F0B493A662EDAB57E130C914 (OK: length 16)
 Fingerprints                 SHA1 9FF7AAF0607D74D55F356BD34A0E32BDE6380469
                              SHA256 E2604001FD2BF9BA2068CE0392A194DA4027E260968FF5C87E813E055EE6BFB1
 Common Name (CN)             buildstaging.com  (request w/o SNI didn't succeed)
 subjectAltName (SAN)         buildstaging.com *.play.buildstaging.com *.hotpay.buildstaging.com *.buildstaging.com
                              *.saas.buildstaging.com *.whatsapp.buildstaging.com *.dsp.buildstaging.com *.sites.buildstaging.com
                              *.analytics.buildstaging.com *.data.buildstaging.com *.hotwallet.buildstaging.com
                              *.club.buildstaging.com *.display.buildstaging.com *.security.buildstaging.com *.bkf.buildstaging.com
                              *.integration.buildstaging.com *.content.buildstaging.com *.backoffice.buildstaging.com
                              *.devops.buildstaging.com *.auth.buildstaging.com *.send.buildstaging.com *.cp.buildstaging.com
                              *.cb.buildstaging.com *.hp.buildstaging.com *.sparkle.buildstaging.com *.vulcano.buildstaging.com
                              *.secops.buildstaging.com *.klickpages.buildstaging.com *.spk.buildstaging.com
```

Figure 4.19.4: Certificate details (valid chain, SAN) – Screenshot of "Certificate" + "Chain of trust" + "EV cert

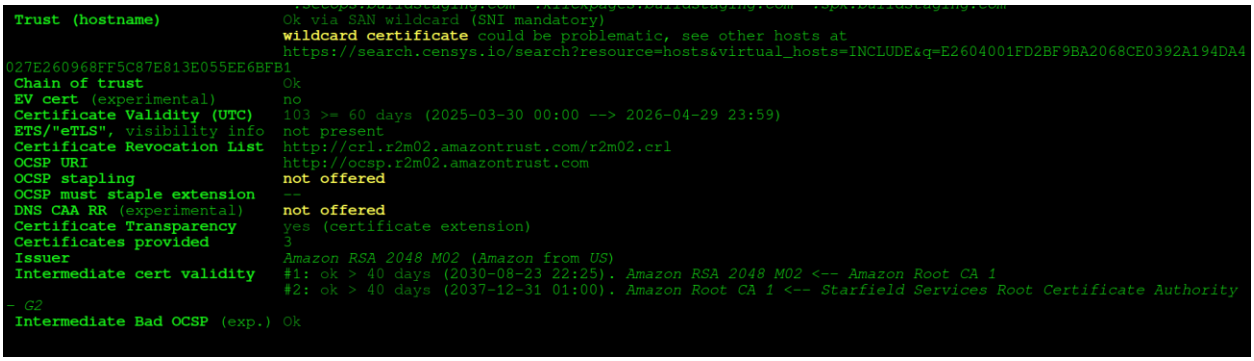


Figure 4.19.5: Certificate details (valid chain, SAN) – Screenshot of "Certificate" + "Chain of trust" + "EV cert".

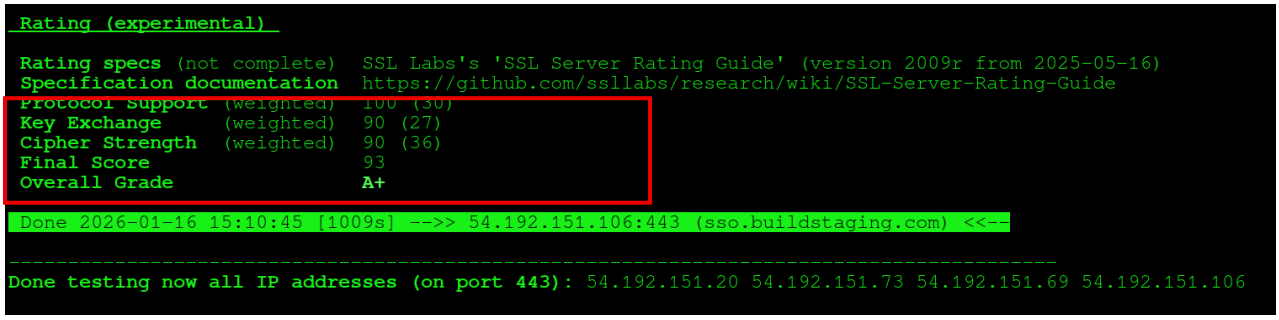


Figure 4.19.6: Rating & Grade (A+) – Screenshot of "Rating (experimental)" + "Overall Grade".

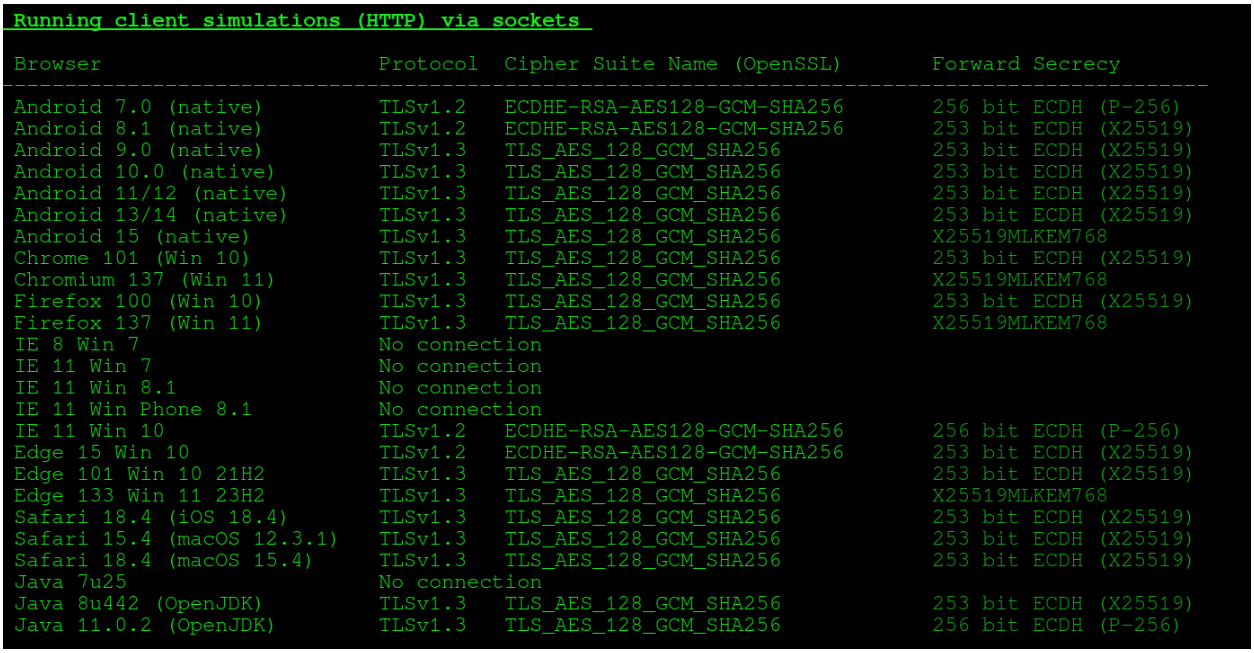
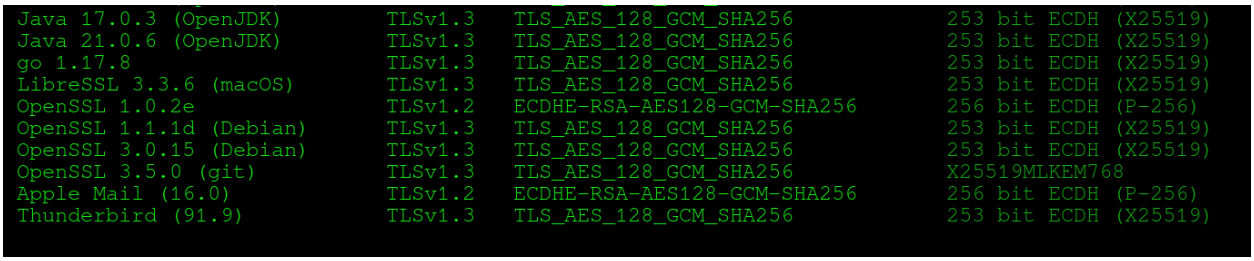


Figure 4.19.7: Client simulations (modern browsers OK) – Screenshot of "Running client simulations".



Java 17.0.3 (OpenJDK)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Java 21.0.6 (OpenJDK)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
go 1.17.8	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
LibreSSL 3.3.6 (macOS)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
OpenSSL 1.0.2e	TLSv1.2	ECDHE-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
OpenSSL 1.1.1d (Debian)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
OpenSSL 3.0.15 (Debian)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
OpenSSL 3.5.0 (git)	TLSv1.3	TLS_AES_128_GCM_SHA256	X25519MLKEM768
Apple Mail (16.0)	TLSv1.2	ECDHE-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Thunderbird (91.9)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)

Figure 4.19.8: Client simulations (modern browsers OK) – Screenshot of "Running client simulations".

Status: Not exploitable / Mitigated

4.20 CWE 287: Improper Authentication

The authentication endpoint (sso.buildstaging.com/login with OAuth service parameter) was tested for improper authentication vulnerabilities, including parameter tampering, provider confusion, weak password handling, session fixation, and privilege escalation attempts.

Tested Scenarios:

- Added privileged parameters: role=admin, bypass=true, isAdmin=true in POST body.
- Provider manipulation: providerSelected=Google, providerSelected=ADMIN.
- Username tampering: admin@hotmart.com (privileged account attempt).
- Password policy bypass: Blank password, special characters only.
- Session fixation: Removed JSESSIONID cookie from request.
- All requests used fresh execution and _csrf values captured from legitimate failed login attempts.

Key Observations:

- Every variation resulted in HTTP 202 Accepted with X-Amzn-Waf-Action: challenge header and a CloudFront WAF JavaScript challenge page.
- No request resulted in successful authentication (200 OK, redirect to dashboard, or admin access).
- No 401 Unauthorized or custom error messages indicating parameter acceptance.
- WAF consistently triggered on suspicious parameter combinations and cookie manipulation, preventing any authentication bypass or privilege escalation.
- No evidence of weak password policy bypass, provider confusion, session fixation, or parameter-based authentication bypass.
- CloudFront WAF effectively mitigated all tampering attempts by enforcing a challenge before processing.

Impact: None observed — no authentication bypass or unauthorized access achieved.

PoC:

Figure 4.20.1: role=admin & bypass → true → 202 WAF challenge

Figure 4.20.2: role=admin & bypass → true → 202 WAF challenge

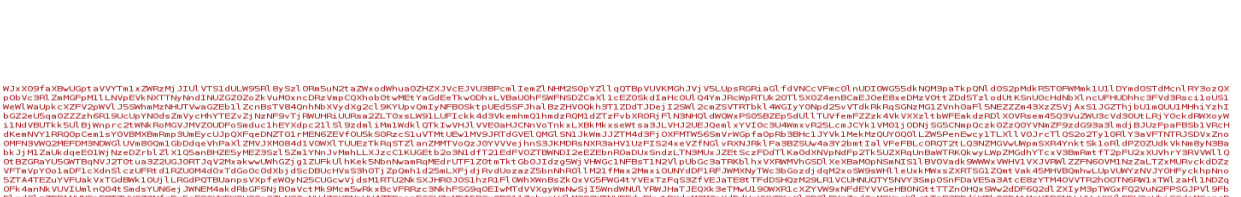


Figure 4.20.3: role=admin & bypass → true → 202 WAF challenge

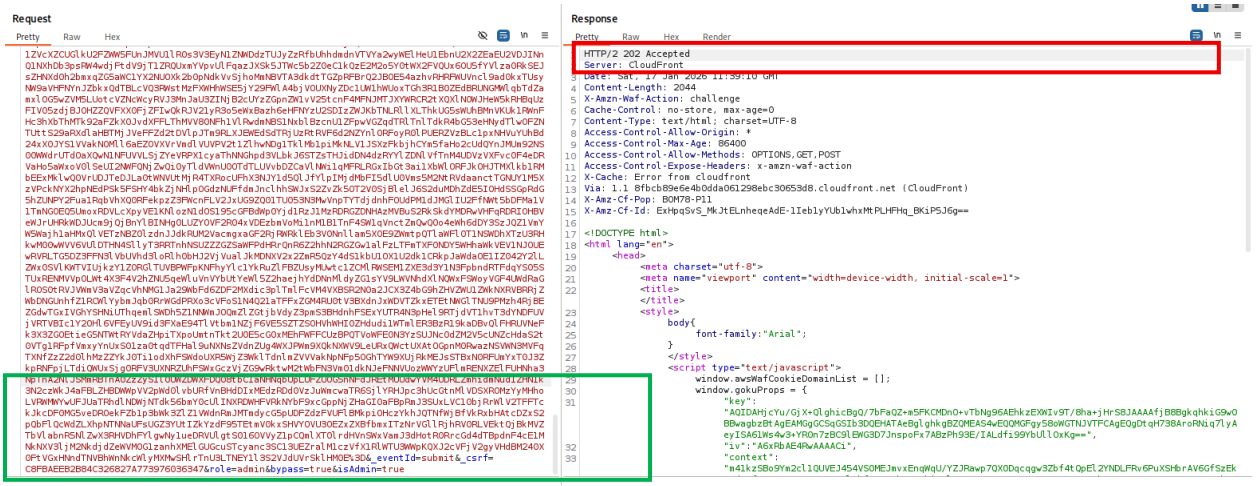


Figure 4.20.4: role=admin & bypass=true & isAdmin=true → 202 WAF challenge

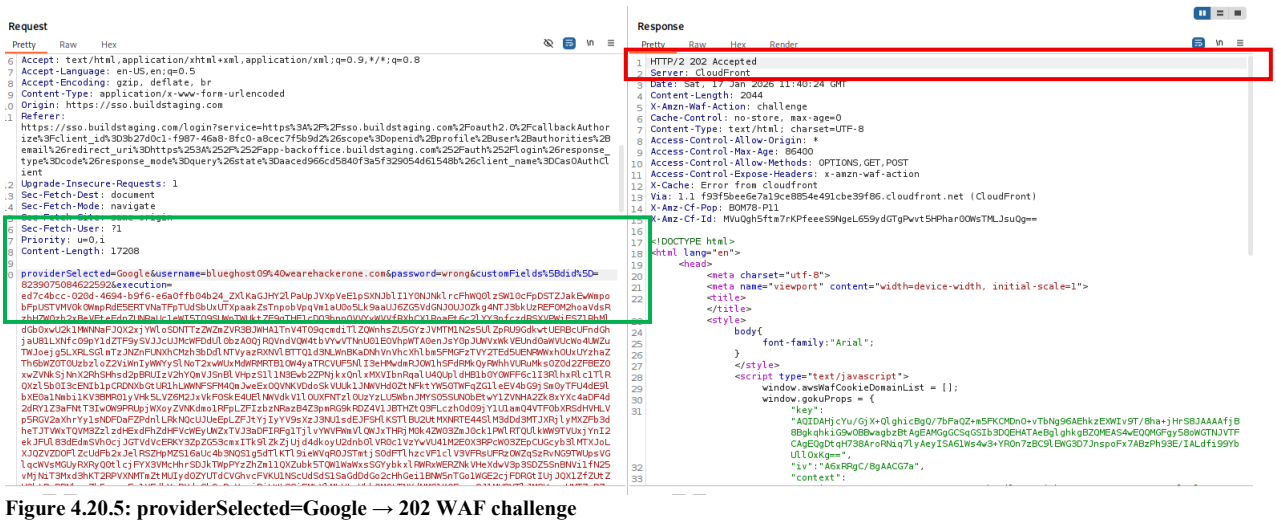


Figure 4.20.5: providerSelected=Google → 202 WAF challenge

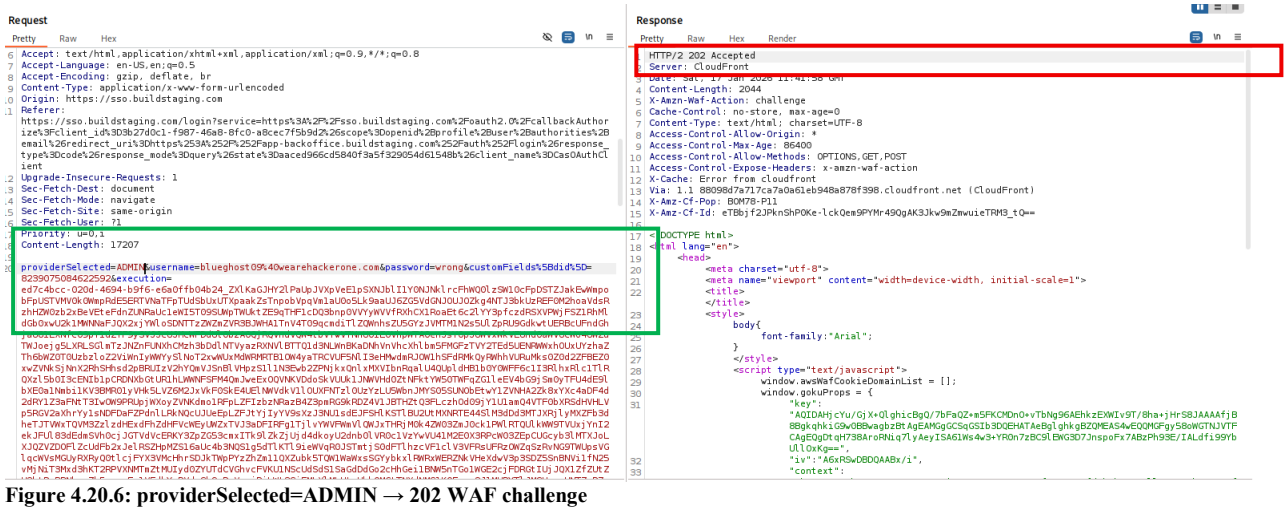


Figure 4.20.6: providerSelected=ADMIN → 202 WAF challenge

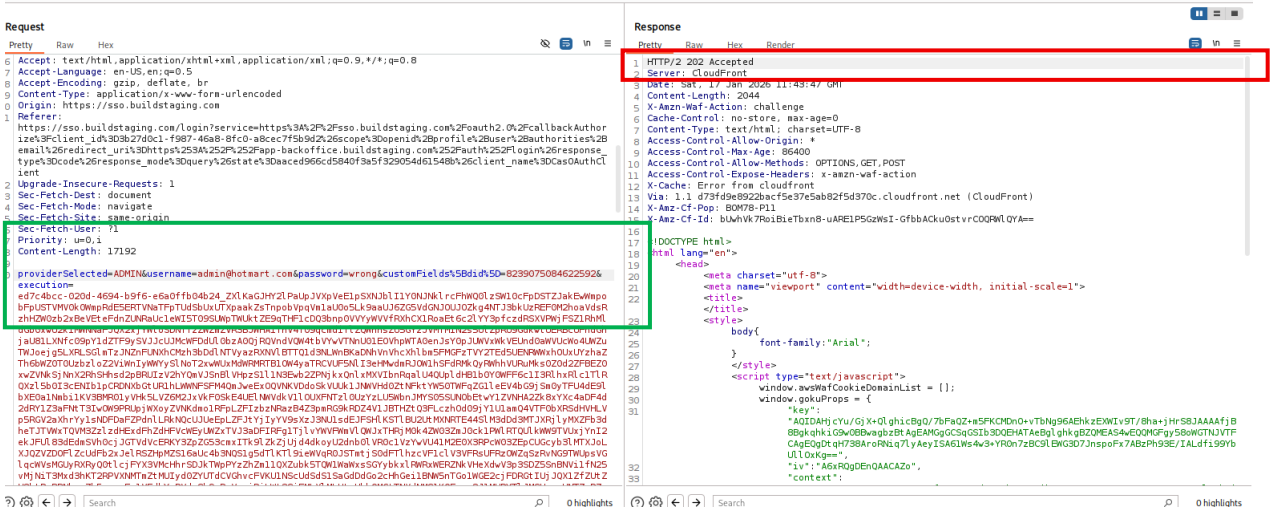


Figure 4.20.7: username=admin@hotmart.com → 202 WAF challenge

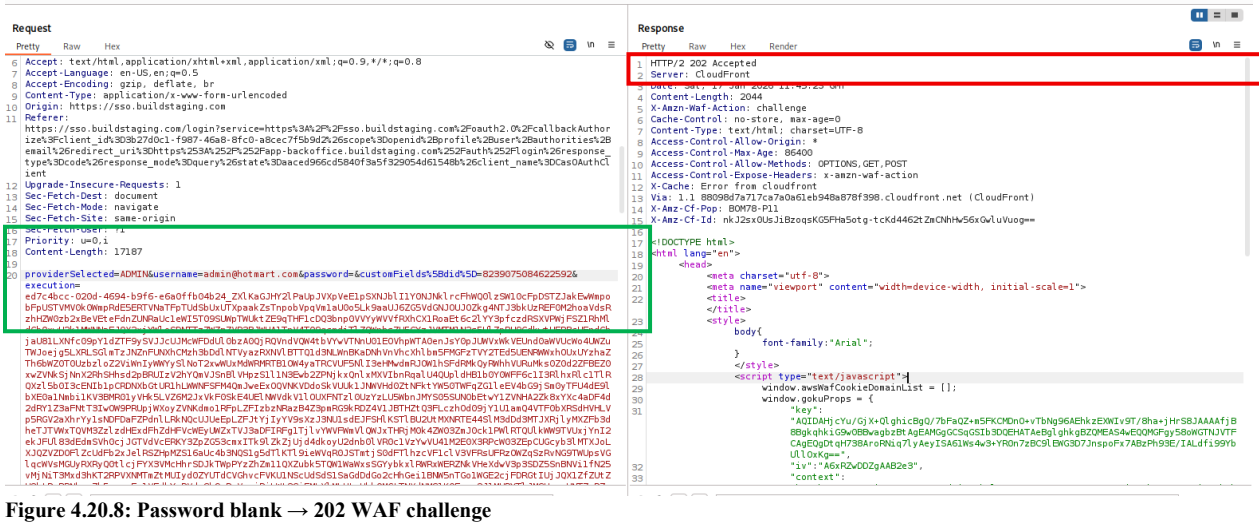


Figure 4.20.8: Password blank → 202 WAF challenge

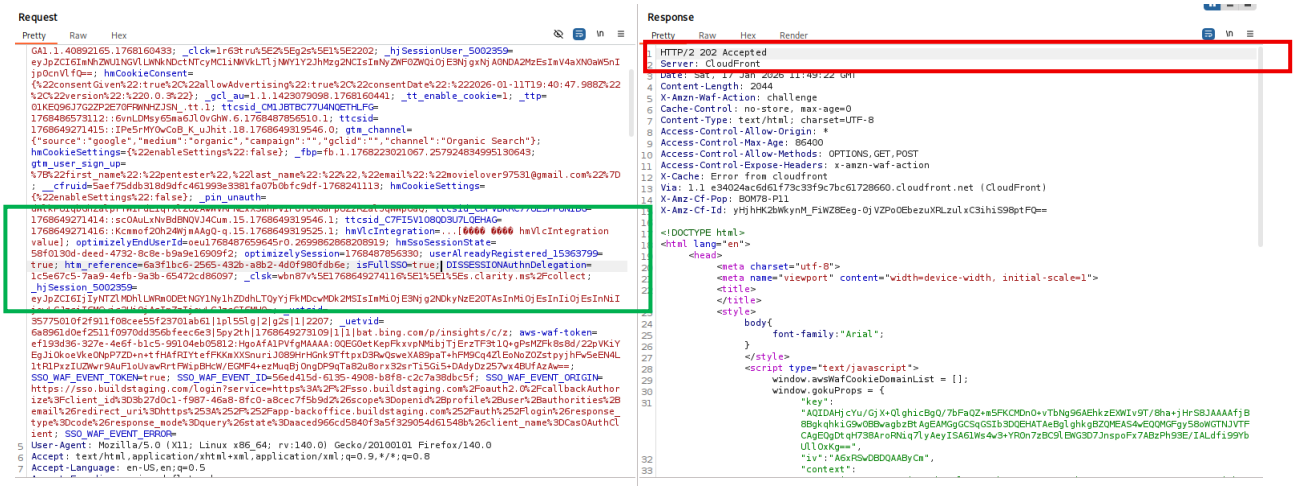


Figure 4.20.9: JSESSIONID removed → 202 WAF challenge

Status: The authentication mechanism is robust and does not allow bypass via parameter tampering, provider confusion, weak credential submission, or session manipulation. All suspicious requests are intercepted by CloudFront WAF, ensuring no improper authentication occurs.

5. Conclusion

Team Blue Diamond has completed a focused security assessment of the *.buildstaging.com environment, prioritizing Authentication and Session Management. Our methodology combined automated reconnaissance with deep manual testing to simulate real-world attack vectors. The assessment indicates that the infrastructure maintains a strong security posture, largely due to effective WAF implementation. However, to ensure a robust defense-in-depth strategy, we recommend addressing the identified configuration improvements outlined in this report.